15 直すべき所と直し方（改修の一覧）

> この巻は、業務工数システムのプログラム（GitHub の `master`。2026 年 4 月 17 日のコミット `30d45e7`）を 1 ファイルずつ読んで見つけた「直すべき所」を、**全部** 並べた物です。1 つずつ、次の 4 つを書いています。
>
> - **どこ:** ファイルと行番号（`master` の行番号です。手元のファイルを書き換えていると、ずれることがあります）
> - **今どうなっていて、何が困るか**
> - **どう直すか**（直した後のコード）
> - **直ったことの確かめ方**
>
> 🔰 **初めての方へ:** 全部をいっぺんに直す必要はありません。§2 の一覧で「🔴 すぐ」の物から、1 つずつ直してください。直し方のコードは、そのまま貼れば動くように書いてあります。ただし **貼る前に、必ず作業用のブランチを作ってください**（§3）。`master` に入れた物は、そのまま本番に出ます（000_引き継ぎ書_最初に読む の §6）。

この巻で分かること

- 本番のデータが守られていない所（ログインや権限の確かめ漏れ）と、全部の API にまとめて確かめをかける直し方
- 動かない所・壊れやすい所（不具合）と、その直し方
- リポジトリ・デプロイ・部品（ライブラリ）の整理のしかた
- 直した後に足すテストと、直す順番

目次

1. この巻の読み方
2. まず、全体を 1 枚で（全部の一覧）
3. 直す前の準備
4. 安全の問題（S）
5. 不具合（B）
6. 開発・運用の整理（D）
7. テスト（T）
8. 直す順番（おすすめ）
9. この巻で確かめたこと
10. この巻のまとめ

---

1. この巻の読み方

1.1 印の意味

急ぐ度

印	意味
🔴 すぐ	本番のデータが守られていない。ほかの作業より先に直す
🟠 早めに	動かない、または壊れやすい。使う人が困る
🟡 余裕のある時に	整理・改善。直さなくても、すぐには困らない

どうやって分かったか

印	意味
✅ 手元で再現	`master` を手元の別のフォルダに書き出し、メモリの中だけのデータベースで動かして、本当にそうなることを確かめた（本番には一切触っていない。§9）
📖 コードから	コードを読んで分かった。動かしては確かめていない

番号の頭の文字: S ＝ 安全（Security）、B ＝ 不具合（Bug）、D ＝ 開発・運用の整理（Development）、T ＝ テスト（Test）。

1.2 この巻で使う言葉

言葉	意味
API（エーピーアイ）	画面（React）がサーバー（Django）に「データをちょうだい」「これを保存して」と頼む窓口。`/api/member_new/` のような URL の 1 つ 1 つ（07 巻）
ログインの確かめ（認証）	「今、だれがログインしているか」を確かめること。このアプリでは、ログインすると **セッション**（サーバーが覚えておく、その人用のメモ）に従業員番号（`login_No`）が入る
権限の確かめ（認可）	「その人に、その操作をしてよいか」を確かめること。このアプリでは、人員の表の「権限（`authority`）」と「管理者（`administrator`）」の 2 つのスイッチ（000 巻 §2）
CSRF（シーサーフ）	ログインしたままの人が、別の（悪い）サイトを開いた時に、そのサイトが裏でこのアプリへ「消して」などを送らせる攻撃。防ぐには、このアプリの画面だけが知っている合言葉（**CSRF トークン**）を、書き込みの時に一緒に送らせて確かめる
CORS（コルス）	別のサイトの JavaScript が、このアプリの API の返事を読んでよいかを、サーバーが答える仕組み
Cookie（クッキー）	ブラウザがサイトごとに覚えておく小さなデータ。ログインのセッションの番号も、CSRF トークンもここに入る
ステータスコード	API の返事に付く 3 けたの番号。200 ＝ 成功、400 ＝ 送った物がおかしい、401 ＝ ログインしていない、403 ＝ してはいけない、404 ＝ 見つからない、500 ＝ サーバーの中でエラー（09 巻）

2. まず、全体を 1 枚で（全部の一覧）

番号	急ぐ度	何が困るか（ひとことで）	主な場所	分かり方
S-1	🔴	ログインの確かめを API ごとに書いていて、書き忘れが多い（直し方の土台）	`kosu/permissions.py`（新しく作る）・`settings.py`	✅
S-2	🔴	ログインしていなくても使える書き込みの API が 30 個以上ある	`kosu/views/` の各ファイル	✅
S-3	🔴	ログインした人なら、他人の工数・問い合わせを見たり、変えたり、消したりできる	`kosu_views.py`・`team_views.py`・`inquiry_views.py`	✅
S-4	🔴	ダウンロードの API が、渡された場所のファイルを、確かめずに返して、後で消す	`asynchronous_views.py` 242〜263 行目	✅
S-5	🔴	どのサイトからでも、ログイン中の人として操作でき、返事も読める（CORS・Cookie・CSRF）	`settings.py` 53〜59・76〜77 行目	✅
S-6	🔴（要相談）	ログインが従業員番号だけで、パスワードが無い。セッションは 10 年	`main_views.py` 79〜102 行目・`settings.py` 74 行目	📖
S-7	🔴	ヘルプ画面の「全部消す」API が、パスワードを確かめていない	`main_views.py` 106〜189 行目	✅
S-8	🔴	サーバーのログを、ログインしていなくても読める	`main_views.py` 935〜953 行目	✅
S-9	🔴	リポジトリに、氏名入りのバックアップの Excel が記録されている	リポジトリの直下	📖
B-1	🟠	ヘルプ画面の初期化が、URL の `/` が 1 つ多くて動かない	`Help.tsx` 57 行目	✅
B-2	🟠	バックアップのダウンロードの URL に `//` ができる（本番で動いていない恐れ）	`AdministratorLoading.tsx` 40 行目・`.env.production`	✅
B-3	🟠	定義の Ver 名を変える・消すと、その Ver で入れた工数が開けなくなる	`def_views.py` 164〜208 行目	✅
B-4	🟠	班員に入れた人を人員から消すと、班長の画面が 500	`team_views.py` 295〜297・447〜449 行目	✅
B-5	🟠	人員から消した人の問い合わせが、開けない・直せない・消せない	`inquiry_views.py` 160〜163 行目など	✅
B-6	🟠	同じ従業員番号の人が 2 人いると、その人の画面がほとんど 500	`models.py` 23 行目	✅

番号	急ぐ度	何が困るか（ひとことで）	主な場所	分かり方
B-7	🔴	設定データが無いと、多くの画面が 500	`main_views.py` 224〜227 行目など 10 か所	📖
B-8	🔴	データのロードが途中で失敗すると、データが半分だけ消える	`tasks.py`	📖
B-9	🟡	ログインが切れていると、401 ではなく 500 になる API がある	`kosu_views.py` 148 行目など	📖
B-10	🔴	ログイン以外の理由でも 401 を返し、画面がログイン画面へ飛ぶ	29 か所	✅
B-11	🟡	作業詳細選択肢の「ロード開始」が、定義のロード中に「実行中…」になる	`AdministratorLoading.tsx` 494 行目	📖
B-12	🟡	ファイル選びで CSV・XLS も選べるが、サーバーは xlsx しか読めない	`AdministratorLoading.tsx` 398 行目など	📖
B-13	🔴	バックアップ・ロードがサーバーの再起動で止まり、「待ち」のまま残る	`asynchronous_views.py` 26・191 行目	📖
B-14	🟡	月で絞り込むと、月初めの 0〜9 時の記録が前の月に入る	`main_views.py` 748・858 行目	📖
B-15	🟡	思っていない値が送られると 500 になる	`kosu_views.py` 1222 行目など	📖
B-16	🟡	月間・年間の集計で、最初の 1 件の定義 Ver で全部を数える	`kosu_views.py` 1249〜1257 行目	📖
B-17	🟡	操作履歴を 1 件保存するたびに、全件を数えている（遅くなる）	`models.py` 432〜441 行目	📖
B-18	🟡	問い合わせの通知が、別の人の問い合わせを指すことがある	`inquiry_views.py` 112 行目	📖
B-19	🔴	従業員番号を変えると、その人の工数・班・問い合わせが切り離される	`member_views.py` 188〜203 行目	📖
B-20	🔴	`.gitattributes` のせいで、Excel・フォントなどのファイルが壊れる	`.gitattributes` 1〜4 行目	✅
B-21	🟡	Excel の見出し「ポップアップID6」の中身が ID5	`tasks.py` 275・341・903・979 行目	📖

番号	急ぐ度	何が困るか（ひとことで）	主な場所	分かり方
D-1	🟠	GitHub Actions が、テストを流さずに本番へ出している	`.github/workflows/`	📖
D-2	🟠	本番のデータベースの migrate を、だれが・どこで打つか決まっていない	（手順）	📖
D-3	🟡	リポジトリに、入れなくてよい物が 1,200 ファイル以上ある	`staticfiles/` など	📖
D-4	🟡	手元で動かす時に、2 か所を書き換える必要がある	`settings.py`・`.env.production`	📖
D-5	🟡	住所（ホスト名）がコードに直接書いてある	`settings.py` 19〜23・162〜166 行目	📖
D-6	🟡	HTTPS まわりの安全設定が足りない	`settings.py`	📖
D-7	🟡	使っていない部品が 10 個以上入っている	`requirements.txt`・`settings.py`	📖
D-8	🟠	サポート（安全の直し）が終わった部品を使っている	`requirements.txt`・`package.json`	📖
D-9	🟡	`print` の行き先のすり替えと、ログのファイルの場所	`main_views.py` 27〜38 行目	📖
D-10	🟡	古い入口 `/BA…` から、全部の API が使える	`hozen_another/urls.py` 11 行目	✅
D-11	🟡	使っていないコードが残っている	`tasks.py`・`models.py`・`main_views.py`	📖
D-12	🟡	同じコードの繰り返しが多い	各所	📖
D-13	🟡	URL の名前の入れ違い・末尾の `/` のばらつき	`kosu/urls.py`	📖
T-1	🔴	直すと落ちるテストが 8 個（危ない動きを「正しい」と確かめている物もある）	`kosu/tests/test_backend.py`	✅
T-2	🟠	直した所を守るテストを足す	`kosu/tests/`	✅

3. 直す前の準備

3.1 作業用のブランチを作る

`master` に push すると、GitHub Actions が数分で本番に出します（000 巻 §6、09 巻 §14）。直している途中の物が本番に出ないように、**必ず作業用のブランチで直します。**

コピー

```
cd hozen-kosu-React
git switch master
git pull
git switch -c fix/security-step1
```

直し終わったら、ブランチを GitHub に push し、GitHub の画面で **Pull Request**（「このブランチを master に入れてください」というお願い）を作ります。前任者か、もう 1 人に見てもらってから master に入れる（Merge）と、その時に本番に出ます（02 巻）。

3.2 本番のデータを、先にバックアップする

直した物を本番に出す前に、本番のデータを残しておきます。

- **Azure のバックアップ**: 本番のデータベース（Azure Database for PostgreSQL）には、Azure が自動で取るバックアップがあります。何日分残っているか、戻す手順を、前任者に確かめておきます（000 巻 §10）。
- **データ管理画面のバックアップ**: 000 巻 §8.2 の 9 種類。ただし、ダウンロードが本番で動いていない恐れがあります（B-2）。ダウンロードしたファイルが Excel で開けるかを、必ず確かめてください。

3.3 今のテストが全部通ることを確かめる

直す前に、今のテストが通ることを確かめます（12 巻）。

コピー

```
python manage.py test --settings=hozen_another.test_settings
```

2026 年 11 月 9 日に `master` の写しで流した時は、**331 個すべて OK** でした（最後に `Ran 331 tests` と `OK` が出ます）。途中に `database table is locked` という長いエラーの文字が出ますが、テストの結果には関係しません（バックアップのテストが裏で動かすスレッドが、テスト用のデータベースを触ろうとして出る物です）。

画面側のテスト（`frontend/src/tests/` の 65 ファイル）も流しておきます。こちらは今回、流していません。

コピー

```
cd frontend
npm run test:run
```

3.4 1 つ直したら、すぐ確かめて、コミットする

- 1 つの番号（例: S-4）を直したら、テストを流し、手元でその画面を動かしてから、コミットします。
- コミットのメッセージに番号を書いておくと、後で探せます（例: `S-4: ダウンロードを media の中のファイルだけにする`）。
- いくつもの番号を 1 つのコミットに混ぜないでください。何かが壊れた時に、どれのせいか分からなくなります。

---

4. 安全の問題（S）

4.0 何が起きているか

このアプリの API は、Django REST framework の `APIView`（07 巻）で書かれています。`APIView` は、何も書かなければ **だれからの頼みでも受け付けます**。そこで、今は API ごとに、最初の数行で「ログインしているか」「権限があるか」を自分で確かめています。

コピー

```
# 今の書き方（例: kosu/views/member_views.py 15〜34 行目の MemberList。短くまとめ、説明のコメントを足した物）
login_no = request.session.get('login_No')       # セッションから、ログイン中の人の従業員番号を出す
if not login_no:                                  # 無ければ「ログインしていない」
  return Response({'status': 'error', 'message': 'ログイン情報が確認できません。'}, status=status.HTTP_401_UNAUTHORIZED)
member_data = member.objects.get(employee_no=login_no)
if not member_data.authority:                     # 「権限」が無ければ断る
  return Response({'status': 'error', 'message': 'アクセス権限がありません。'}, status=status.HTTP_403_FORBIDDEN)
```

この書き方だと、**書き忘れた API は、だれでも使えます**。実際に、画面を表示するための `get` には書いてあるのに、保存・削除の `post`・`put`・`delete` に書き忘れている API がたくさんあります（S-2）。画面はボタンを出さないので気付きにくいのですが、API は画面を通さなくても呼べます。

さらに、Django REST framework は、このアプリのような「セッションに番号を入れるだけ」のログインの時には、CSRF の確かめをしません（S-5）。

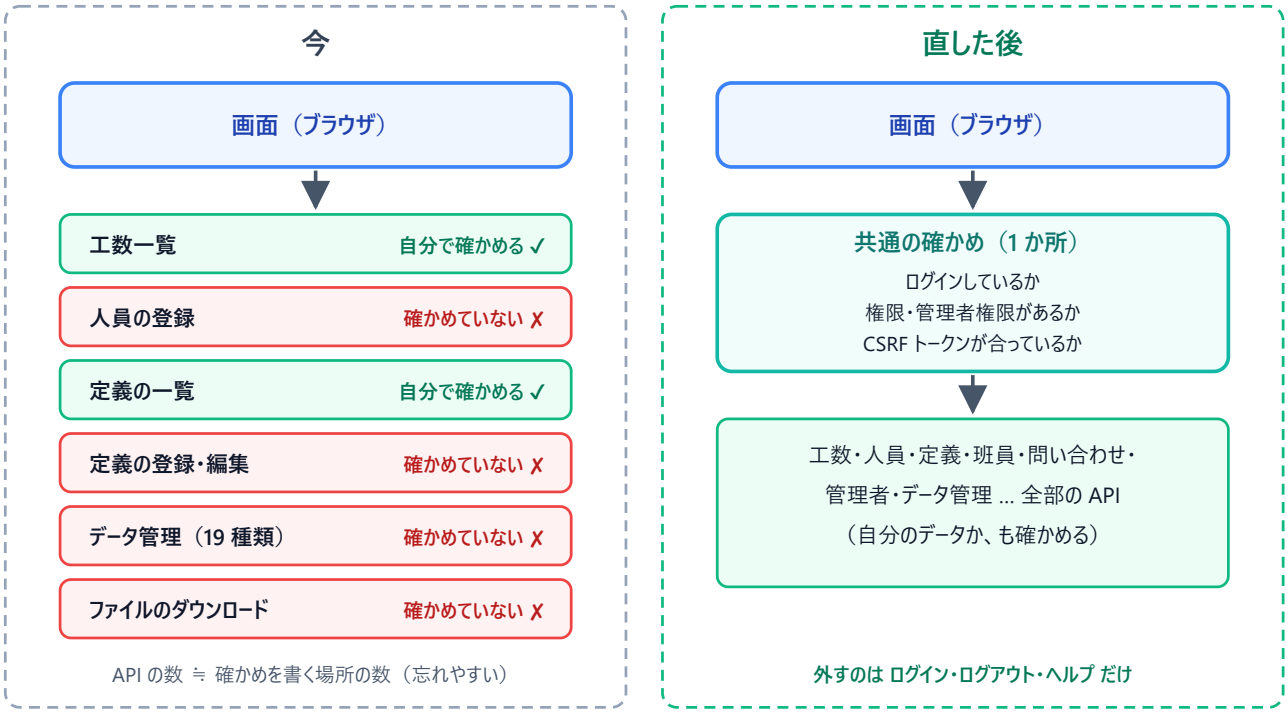


図1:「ログインしているか・権限があるか」を確かめる場所。今は API ごとに自分で確かめていて、書き忘れた API（赤）は、だれでも使える。直した後は、全部の API の手前に共通の確かめを 1 つ置き、要らない所（ログイン画面など）だけ外す

直し方の方針は、図の右側です。「全部の API の手前で、まず確かめる」部品を 1 つ作り、**最初から全部の API にかけて**、要らない所（ログイン画面など）だけ外します（S-1）。こうすると、新しい API を足した時に書き忘れても、安全な側（ログインが要る）になります。

4.1 S-1 共通の確かめを作り、全部の API に最初からかける

🔴 すぐ ／ ✅ 手元で再現（この節のコードを `master` の写しに入れ、§9 の確かめが全部通ることを確かめた）

どこ: 新しく作る `kosu/permissions.py`、`hozen_another/settings.py` の 61〜70 行目（`REST_FRAMEWORK`）、`kosu/models.py` の 64〜65 行目（`member` の終わり）

直し方:

① `kosu/permissions.py` を新しく作ります。

コピー

```
"""ログイン中の人と、その人の権限を確かめる部品（全部の API で共通に使う）。

settings.py の REST_FRAMEWORK で「最初から全部の API にかける」設定にするので、
ログインが要らない API（ログイン・ログアウト・ヘルプ）だけ AllowAny を付けて外す。
"""
from rest_framework import exceptions
from rest_framework.authentication import BaseAuthentication, CSRFCheck
from rest_framework.permissions import BasePermission

from .models import member


class LoginNoAuthentication(BaseAuthentication):
  """セッションの login_No から、ログイン中の人（member）を探す。"""

  def authenticate(self, request):
    login_no = request._request.session.get('login_No')
    if not login_no:
      return None                       # ログインしていない → request.user は None
    login_member = member.objects.filter(employee_no=login_no).first()
    if login_member is None:
      return None                       # 人員から消された人 → ログインしていない扱い
    self.enforce_csrf(request)          # ログイン中の人の書き込みは、CSRF トークンも確かめる
    return (login_member, None)

  def enforce_csrf(self, request):
    # DRF の SessionAuthentication.enforce_csrf と同じ確かめ方
    def dummy_get_response(request):
      return None
    check = CSRFCheck(dummy_get_response)
    check.process_request(request)
    reason = check.process_view(request, None, (), {})
    if reason:
      raise exceptions.PermissionDenied(f'CSRF Failed: {reason}')

  def authenticate_header(self, request):
    # これを返すと、ログインしていない時に 403 ではなく 401 になる（画面はログイン画面へ戻る）
    return 'Session'


class IsLoggedIn(BasePermission):
  """ログインしていること。"""
  def has_permission(self, request, view):
    return isinstance(request.user, member)


class HasAuthority(IsLoggedIn):
  """ログインしていて、「権限」が付いていること（人員・班員の画面）。"""
  def has_permission(self, request, view):
    return super().has_permission(request, view) and bool(request.user.authority)


class IsAdministrator(IsLoggedIn):
  """ログインしていて、「管理者権限」が付いていること（管理者・定義の編集の画面）。"""
```

```
  def has_permission(self, request, view):
    return super().has_permission(request, view) and bool(request.user.administrator)
```

▶ ここで初めて出てくるコード

- `.process_request()`（Django）: Django のミドルウェア（頼みの前後に動く部品）の、頼みが来た時に動くメソッド。`CSRFCheck` では、クッキーから CSRF の合言葉を読み出します。
- `.process_view()`（Django）: Django のミドルウェアの、画面（ビュー）を動かす直前に動くメソッド。`CSRFCheck` では、送られてきた合言葉が合っているかを確かめ、合わなければ理由の文を返します。
- `.PermissionDenied()`（Django REST framework）: `exceptions.PermissionDenied('…')` は、「してはいけない」というエラーを起こす物。Django REST framework が受け止めて、403 の返事にします。
- `bool()`（Python）: 値を `True`・`False` にする関数。空の文字列や `None` は `False`、中身があれば `True` です。
- `BaseAuthentication`（Django REST framework）:「この頼みは、だれからか」を調べる部品の土台。これを受け継いで `authenticate` を書き、だれか分かったら `(その人, None)`、分からなければ `None` を返します。
- `CSRFCheck`（Django REST framework）: Django の CSRF の確かめ（`CsrfViewMiddleware`）を、Django REST framework の中から使うための部品。`rest_framework.authentication` から読み込みます。
- `BasePermission`（Django REST framework）:「この人に、この API を使わせてよいか」を決める部品の土台。これを受け継いで `has_permission` を書き、よければ `True` を返します。`False` なら 401 か 403 になります。
- `.user`（Django）: Django の `request.user` は、今ログインしている人（していなければ「だれでもない人」）です。`request.user.is_authenticated` で、ログインしているかが分かります。

② `hozen_another/settings.py` の `REST_FRAMEWORK`（61〜70 行目）の最後に、3 行足します。

コピー

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20,
    'DEFAULT_PARSER_CLASSES': (
        'rest_framework.parsers.JSONParser',
    ),
    'DEFAULT_RENDERER_CLASSES': (
        'rest_framework.renderers.JSONRenderer',
    ),
    'DEFAULT_AUTHENTICATION_CLASSES': ['kosu.permissions.LoginNoAuthentication'],  # だれがログインしているか
    'DEFAULT_PERMISSION_CLASSES': ['kosu.permissions.IsLoggedIn'],                 # 最初から全部の API でログインが要る
    'UNAUTHENTICATED_USER': None,                                                  # ログインしていない時の request.user は None
}
```

③ `kosu/models.py` の `member` の終わり（64〜65 行目の `__str__` の下）に、3 行足します。Django REST framework の一部（回数の制限など）が `request.user.is_authenticated` を見るためです。データベースの列ではないので、migrate は要りません。

コピー

```
  def __str__(self):
    return self.name

  @property
  def is_authenticated(self):          # ログイン中の人として扱われた時は、いつも True
    return True
```

▶ ここで初めて出てくるコード

- **`@property`**（Python）：クラスのメソッドの上に付けると、`()` 無しで値のように読めるようにする印。

④ ログインが要らない 4 つの API に、`permission_classes = [AllowAny]` を付けます（`kosu/views/main_views.py`）。

クラス	行	なぜ要らないか
`Login`	79 行目	ログインする所なので
`Help`	106 行目	ログインできなくなった時に使う所なので（代わりにパスワードを確かめる。S-7）
`HelpPass`	176 行目	同上
`Logout`	194 行目	ログインが切れていても、ログアウトは通してよい

コピー

```
from rest_framework.permissions import AllowAny     # main_views.py の import に足す

class Login(APIView):
  permission_classes = [AllowAny]    # このクラスだけ、ログインの確かめを外す
  # POST処理
  def post(self, request):
    ...
```

▶ ここで初めて出てくるコード

- **`AllowAny`**（Django REST framework）：「だれでも使える」という確かめ（いつも通す）。全部の API に「ログインが要る」をかけた後で、ログイン画面のように、ログインしていなくても使う API にだけ `permission_classes = [AllowAny]` と付けて外します。

⑤「権限」「管理者権限」が要る API に、`HasAuthority` か `IsAdministrator` を付けます。どれに何を付けるかは S-2 の表にあります。

コピー

```
from ..permissions import HasAuthority, IsAdministrator

class MemberNew(APIView):
  permission_classes = [HasAuthority]      # 「権限」が無い人は 403
```

1 つのクラスの中で、`get` と `put` で要る権限が違う時は、`get_permissions` を書きます。

コピー

```
from ..permissions import IsLoggedIn, IsAdministrator

class Example(APIView):
  def get_permissions(self):
    if self.request.method == 'GET':
      return [IsLoggedIn()]              # 見るだけなら、ログインしていればよい
    return [IsAdministrator()]           # 書き込みは管理者だけ
```

⑥ 関数で書いた API（`kosu/views/asynchronous_views.py`）は、`@api_view` のすぐ下に `@permission_classes` を付けます。

コピー

```
from rest_framework.decorators import api_view, parser_classes, permission_classes
from ..permissions import IsAdministrator

@api_view(['POST'])
@permission_classes([IsAdministrator])          # バックアップ・削除・ロードは管理者だけ
@parser_classes([MultiPartParser, JSONParser, FormParser])
def backup(request):
  ...
```

▶ ここで初めて出てくるコード

- **`permission_classes()`**（Django REST framework）：`from rest_framework.decorators import permission_classes` で読み込む、関数の API に確かめを付ける道具。`@permission_classes([…])` の形で使います。クラスの API では、`permission_classes = [...]` という変数に書きます。
- **`@permission_classes`**（Django REST framework）：`@api_view` で書いた関数の API に、「だれが使えるか」の確かめを付ける印。`@permission_classes([IsAdministrator])` で、管理者だけにします。`@api_view` のすぐ下に書きます。

`check_task_status`（219 行目）と `download_file`（242 行目）にも、同じように `@permission_classes([IsAdministrator])` を付けます。

⑦ `WebConsoleLogView`（`main_views.py` 935 行目）は、Django REST framework ではなく Django の `View` で書かれているので、⑤の設定が効きません。`APIView` に変えます（S-8）。

変わること:

- ログインしていない人の頼みは、API の中身が動く前に **401** で断られます（返事は `{"detail": "認証情報が含まれていません。"}`）。権限が無い時は **403** です。画面は今と同じく、401 ならログイン画面へ、403 ならメインメニューへ戻ります。
- 今ある「ログインしているか」の数行は、残しておいても動きます（2 回確かめるだけ）。全部直し終わってから、少しずつ消すと読みやすくなります（D-12）。
- ログインが切れている時に 500 になっていた API も、401 になります（B-9）。

確かめ方: T-1 の 8 個を直し、T-2 のテストを足して、全部通ること。§9 の表の「直した後」と同じになること。

4.2 S-2 ログインや権限を確かめていない API（全部）

🔴 すぐ ／ ✅ 手元で再現（表の「✅」の行）

S-1 の②で、全部の API に「ログインが要る」がかかります。そのうえで、下の表の「付ける確かめ」を、それぞれのクラスに付けます。

ログインもしていない人が使える API（今）

API（/api/…/）	方法	場所	今どうなっているか	付ける確かめ
member_new	POST	member_views.py 85 行目	だれでも人員を登録できる。「管理者」の人も作れて、その番号でログインできる ✅	HasAuthority ＋S-3 の⑤
member_update/<番号>	PUT	member_views.py 188 行目	だれでも、だれの人員でも書き換えられる（管理者にもできる） ✅	HasAuthority ＋S-3 の⑤
member_delete/<番号>	DELETE	member_views.py 216 行目	だれでも人員を消せる	HasAuthority
manager_update	PUT	main_views.py 489 行目	だれでも設定（一覧の件数・問い合わせ担当者）を変えられる ✅	IsAdministrator
manager_kosu_update/<id>	PUT・DELETE	main_views.py 683・708 行目	だれでも、だれの工数でも書き換え・削除できる	IsAdministrator
manager_history_detail/<id>	DELETE	main_views.py 923 行目	だれでも操作履歴を消せる ✅	IsAdministrator
web_console_log	GET	main_views.py 935 行目	だれでもサーバーのログを読める ✅	IsAdministrator（S-8）
help	POST	main_views.py 108 行目	パスワードを確かめずに、人員・定義・設定を全部消す ✅	AllowAny ＋パスワード（S-7）
def_ver	POST	def_views.py 34 行目	ログインせずに「使う定義」をセッションに入れられる。ほかの確かめ漏れと組み合わせると、下の team_export が動く ✅	ログイン（自動）＋その Ver があるかの確かめ
def_new	POST	def_views.py 123 行目	だれでも定義を作れる ✅	IsAdministrator
def_update/<id>	PUT	def_views.py 164 行目	だれでも定義を書き換えられる（名前を変えると B-3 が起きる） ✅	IsAdministrator
def_detail_new	POST	def_views.py 290 行目	だれでも作業詳細の選択肢を足せる ✅	IsAdministrator
def_detail_update/<id>	PUT	def_views.py 362 行目	だれでも作業詳細の選択肢を書き換えられる	IsAdministrator

API（`/api/…/`）	方法	場所	今どうなっているか	付ける確かめ
`team_export`	POST	`team_views.py` 347 行目	ショップの人全員の工数入力状況の Excel（氏名入り）を取り出せる ✅	`HasAuthority`
`day_update`	PUT	`kosu_views.py` 747 行目	だれの工数でも、就業日を動かせる ✅	ログイン（自動）＋S-3 の②
`inquir_update/<id>`	DELETE	`inquiry_views.py` 385 行目	だれでも問い合わせを消せる ✅	ログイン（自動）＋S-3 の④
`kosu_backup` ほか 19 個	POST	`asynchronous_views.py` 23 行目	バックアップ・工数の削除・ロード（上書き）を始められる ✅（日付を付けずに送ると「日付を指定してください」が返った ＝ 確かめの前に中身が動いている）	`IsAdministrator`
`check_backup_status`	GET	`asynchronous_views.py` 220 行目	タスクの結果（サーバーの中のファイルの場所）を見られる	`IsAdministrator`
`download_backup`	GET	`asynchronous_views.py` 243 行目	S-4	`IsAdministrator` ＋S-4
`set_day`・`kosu_calendar_change`・`kosu_link`	POST	`kosu_views.py` 301・910・992 行目	セッションに日付・年月を書くだけ（単独では害は小さい）	ログイン（自動）
`team_calendar`・`team_overtime`・`team_view` の POST、`team_calendar_week_jump`・`team_shop_select`	POST	`team_views.py` 318・470・544・326・554 行目	同上	ログイン（自動）

ログインはしているが、「権限」を確かめていない API

API	方法	場所	今どうなっているか	付ける確かめ
`team_new`	POST	`team_views.py` 78 行目	「権限」の無い人も班を登録できる（ログインしていないと 500）	`HasAuthority` ＋S-3 の⑥
`team_detail/<id>`	GET	`team_views.py` 189 行目	「権限」の無い人も、他人の工数の中身を見られる ✅	`HasAuthority` ＋S-3 の③

管理者だけが使う画面の API（`get` の中では確かめているが、クラスに付けておくと書き忘れが無くなる）

`AdministratorUpdate`・`AdministratorKosuUpdate`・`AdministratorHistoryDetail`（`main_views.py`）、`DefNew`・`DefUpdate`・`DefDelete`・`DefList`・`DefDetailNew`・`DefDetailUpdate`・`DefDetailDelete`・`DefDetailList`（`def_views.py`）には `IsAdministrator` を、`MemberList`・`MemberNew`・`MemberUpdate`・`MemberDelete`（`member_views.py`）と `team_views.py` の全部のクラスには `HasAuthority` を付けます。

> 💡 **画面のボタンを隠しても、守ったことにはなりません。** API は、ブラウザの画面を通さなくても、URL に直接送れば動きます。「してよいか」は、必ずサーバー（Django）の側で確かめます（000 巻 §8.5）。

4.3 S-3 他人のデータを見たり、変えたり、消したりできる所

🔴 すぐ ／ ✅ 手元で再現（①の「見る」「消す」、③、④の「回答」）

ログインはしていても、**その行が自分の物か** を確かめていない所です。API の URL の最後の数字（行の id）を変えるだけで、他人の行に届きます。

① 工数の編集・削除（`kosu_views.py`）

API	場所	今
`kosu_update/<id>` の GET・PUT	599・666 行目	他人の工数を見られる ✅・書き換えられる
`kosu_delete/<id>` の DELETE	853 行目	他人の工数を消せる ✅

直し方: 行を取り出した直後に、持ち主を確かめます。他人の行の時は「見つからない（404）」と返します（他人の行がそこにあることも教えないため）。管理者が他人の工数を直す時は、管理者の画面（`manager_kosu_update`）を使います。

コピー

```
# KosuDelete.delete の中（kosu_views.py 861〜863 行目を置き換える）
kosu_instance = self.get_object(pk)
if kosu_instance is None or kosu_instance.employee_no3 != int(login_no):   # 自分の工数でなければ
  return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)
```

`KosuUpdate` の `get` と `put` にも、同じ確かめを入れます。`get` は、607〜615 行目（ログインの確かめ）のすぐ下に入れ、609 行目の「セッションに日付を書く」行は、確かめの後ろに移します（今は、確かめの前に書いている）。`put` は、676 行目（`member_obj` を出す所）の下に入れます。

② 就業日の変更（`day_update`、`kosu_views.py` 745〜770 行目）

今は、送られてきた id の行を、だれの物かを見ずに動かします（766〜769 行目の `update_or_create`）。無い id を送ると、従業員番号の無い行を作ろうとして 500 になります。

コピー

```
# 766〜769 行目を置き換える
kosu_instance = Business_Time_graph.objects.filter(id=request.data.get('id'), employee_no3=login_no).first()
if kosu_instance is None:                       # 無い id・他人の工数
  return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)
kosu_instance.work_day2 = day
kosu_instance.save()
```

③ 班員の工数の詳細（`team_detail`、`team_views.py` 189〜241 行目）

今は、だれの工数でも見られます ✅。自分の班のメンバーの工数だけにします。

コピー

```
# 205 行目（「班員データ確認」）の上に足す
team = team_member.objects.filter(employee_no5=login_no).first()
team_numbers = []
if team:
  team_numbers = [str(getattr(team, f'member{i}')) for i in range(1, 16) if getattr(team, f'member{i}')]
if str(kosu_instance.employee_no3) not in team_numbers:      # 自分の班のメンバーでなければ
  return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)
```

（`login_no` は 196 行目で出し、197〜203 行目でログインを確かめているので、その下に置きます。）

④ 問い合わせの編集・削除（`inquir_update`、`inquiry_views.py` 317〜452 行目）

今は、ログインした人ならだれでも、他人の問い合わせの内容を書き換えたり、回答を書いたりできます ✅。`delete` はログインもいりません ✅。`get`（301〜302 行目）では「質問した人か、問い合わせ担当者か」を確かめているので、同じ確かめを `put` と `delete` にも入れ、さらに **質問した人は内容だけ、担当者は回答だけ** を変えられるようにします。

コピー

```
# put の中。332〜333 行目（data・serializer を作る所）を消し、335〜338 行目（設定データ）の下にこれを置く
staff_numbers = [int(a) for a in [admin_data.administrator_employee_no1, admin_data.administrator_employee_no2,
                                  admin_data.administrator_employee_no3] if a and str(a).strip().isdigit()]
is_owner = int(login_no) == int(inquir_instance.employee_no2)     # 質問した人
is_staff = int(login_no) in staff_numbers                           # 問い合わせ担当者
if not (is_owner or is_staff):
  return Response({'status': 'error', 'message': '権限がありません。'}, status=status.HTTP_403_FORBIDDEN)
allowed = set()
if is_owner:
  allowed |= {'content_choice', 'inquiry'}     # 質問した人は、内容だけ
if is_staff:
  allowed |= {'answer'}                        # 担当者は、回答だけ
data = {key: value for key, value in request.data.items() if key in allowed}
serializer = InquirSerializer(inquir_instance, data=data, partial=True)   # partial=True: 送った項目だけ変える
```

▶ ここで初めて出てくるコード

- `.isdigit()`（Python）：文字列が、全部数字の文字だけでできていれば `True` を返すメソッド。`str(a).strip().isdigit()` で、「従業員番号として読める文字か」を確かめています。
- `.items()`（Python）：辞書の「キーと値」の組を 1 つずつ取り出せる形にするメソッド。`for k, v in d.items():` で、両方を同時に使えます。
- `set()`（Python）：集合（同じ値は 1 つしか入らない入れ物）を作る関数。`seen = set()` で空の集合を作り、`seen.add(x)`・`x in seen` で使います。重複を見つける時に便利です。
- `{キー: 値 for …}`（Python）：`{u.pk: u for u in users}` は、「1 つずつから、キーと値の組を作って、辞書にする」書き方です（辞書内包表記）。

`delete` は、質問した人か担当者（または管理者）だけにします。

⑤「権限」の人が、自分や他人を「管理者」にできる（`member_views.py`）

人員の編集画面（`frontend/src/MemberPage/MemberEdit.tsx` 187〜198 行目）と登録画面（`MemberNew.tsx` 229 行目）には「管理者権限」のスイッチがあり、人員の画面は「権限」の人が使えます。つまり、**「権限」の人は、自分を「管理者」にできます。**サーバーも止めません。

これが決まりどおりか（班長に管理者を任せてよいか）は、前任者・使う人と決めることです。「管理者だけが付け外しできる」にするなら、次のようにします。

コピー

```
def changes_roles(request, data, current=None):
  """管理者でない人が、権限・管理者権限を付けたり外したりしようとしていたら True"""
  if request.user.administrator:
    return False
  for key in ('authority', 'administrator'):
    before = bool(getattr(current, key)) if current else False
    if key in data and bool(data[key]) != before:
      return True
  return False

# MemberNew.post の最初と、MemberUpdate.put の serializer を作る前に
  if changes_roles(request, request.data, member_instance):   # MemberNew では member_instance の所を None に
    return Response({'status': 'error', 'message': '権限・管理者権限は、管理者だけが変えられます。'}, status=status.HTTP_403_FORBIDDEN)
```

画面の側も、ログイン中の人が管理者でない時は、2 つのスイッチを出さない（または押せなくする）ようにします。

⑥ 班員の登録で、だれでも班に入れられる（`team_new` の POST、`team_views.py` 78〜101 行目）

画面（`TeamNew` の `get`、56〜65 行目）では、候補を「同じショップの人（組長以上なら全員）」に絞っていますが、サーバーは送られてきた番号をそのまま保存します。番号が人員にあるか、ショップの決まりに合うかを、保存の前に確かめます。

コピー

```
    # 97 行目の for の前に足す
    login_member = request.user
    for field in [f'member{i}' for i in range(1, 16)]:
      number = data.get(field)
      if not number:
        continue
      target = member.objects.filter(employee_no=number).first()
      if target is None:
        return Response({'status': 'error', 'message': f'{number} は人員にありません。'}, status=status.HTTP_400_BAD_REQUEST)
      if not login_member.shop.startswith('組長以上') and target.shop != login_member.shop:
        return Response({'status': 'error', 'message': f'{number} は別のショップの人です。'}, status=status.HTTP_400_BAD_REQUEST)
```

▶ ここで初めて出てくるコード

- `.startswith()`（Python）：文字列が、その中身で始まっていれば `True` を返すメソッド。`login_member.shop.startswith('組長以上')` は、ショップが「組長以上(P,R,T,その他)」「組長以上(W,A)」のどちらかなら `True` です。

4.4 S-4 ダウンロードの API が、渡された場所のファイルを確かめずに返して、後で消す

🔴 すぐ（いちばん急ぐ） ／ ✅ 手元で再現

どこ: `kosu/views/asynchronous_views.py` 242〜263 行目（`download_file`）、219〜238 行目（`check_task_status`）、`frontend/src/AdministratorPage/AdministratorLoading.tsx` 39〜46 行目、`kosu/tests/test_backend.py` 2455〜2462 行目

今:（間を省き、説明のコメントを足しています）

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
@api_view(['GET'])
def download_file(request):
  file_path = request.GET.get('file_path')          # URL の ?file_path=… をそのまま
  file_handle = open(file_path, 'rb')                # どこのファイルでも開く
  response = FileResponse(file_handle, as_attachment=True, filename=os.path.basename(file_path))
  ...
  response.close = cleanup_on_close                  # 送り終わったら、3 秒後にそのファイルを消す
```

何が困るか:

- ログインの確かめが無く、しかも **ファイルの場所を確かめていません。** バックアップとは関係の無い、サーバーの中のファイルまで渡してしまいます（✅ 手元で、`media` の外のファイルの中身が返りました）。
- 渡した後に、そのファイルを消す処理が付いています。消えてはいけないファイルまで消える恐れがあります（📖 手元の Windows では、ファイルが開いたままなので消えませんでした。本番の Linux では、開いたままでも消せます）。
- `response.close` を丸ごと差し替えているので、本来の `close`（開いたファイルを閉じる処理）が動かず、ファイルが開いたまま残ります。
- 今のテスト `test_download_file`（`test_backend.py` 2455 行目）は、`media` の外の一時ファイルをダウンロードできることを「正しい」として確かめています。

直し方:

① サーバー（`asynchronous_views.py`）：ファイルの **名前だけ** を受け取り、置き場所（`settings.MEDIA_ROOT`）とつなげて、その中にあることを確かめます。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
import os
from pathlib import Path
from django.conf import settings
from django.http import JsonResponse, FileResponse
from rest_framework import status
from rest_framework.decorators import api_view, permission_classes
from ..permissions import IsAdministrator


@api_view(['GET'])
@permission_classes([IsAdministrator])
def download_file(request):
  backup_dir = Path(settings.MEDIA_ROOT).resolve()              # バックアップの置き場所
  file_name = os.path.basename(request.GET.get('file_name', ''))  # 名前だけにする（../ や C:/… を捨てる）
  file_path = (backup_dir / file_name).resolve()
  if not file_name or file_path.parent != backup_dir or not file_path.is_file():
    return JsonResponse({'status': 'error', 'message': 'ファイルが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)

  response = FileResponse(open(file_path, 'rb'), as_attachment=True, filename=file_name)
  original_close = response.close
  def close_then_delete():          # 送り終わったら、先にファイルを閉じてから消す
    original_close()
    try:
      file_path.unlink()
    except OSError:
      pass
  response.close = close_then_delete
  return response
```

▶ ここで初めて出てくるコード

- **`.is_file()`**（Python）：`Path` の `.is_file()` は、その場所にファイルがあれば `True` を返すメソッド。フォルダや、何も無い時は `False` です。
- **`.unlink()`**（Python）：`Path` の `.unlink()` は、そのファイルを消すメソッド。消せない時は `OSError` になります。
- **`original_close()`**（Python）：この巻のダウンロードの直し方で、`FileResponse` が元から持っていた `close`（開いたファイルを閉じる処理）を、変数 `original_close` に取っておいた物。先にこれを呼んでファイルを閉じてから、ファイルを消します。
- **`OSError`**（Python）：ファイルを開く・消すなど、OS とのやり取りで起きるエラーのまとめ役。`except OSError:` で、ファイルが無い・使用中で消せない、などをまとめて受け止めます。

② サーバー（`check_task_status`、231 行目）：サーバーの中の場所（`/home/…/media/…xlsx`）ではなく、名前だけを返します。

コピー

```
    if task.status == 'success':
      return JsonResponse({'status': 'success', 'file_name': os.path.basename(task.result or '')})
```

③ 画面（`AdministratorLoading.tsx` 39〜46 行目）：`<a>` に住所を組み立てて押すのをやめ、`api`（axios）でファイルを受け取ってから保存します。axios は住所のつなぎ方を正しくやるので、B-2 の `//` も起きません。手元で `npm start` の時（D-4）にも動きます。

コピー

```
const handleSuccess = useCallback(async (data: any) => {
  setIsRunning(false);

  if (taskType === 'delete' || taskType === 'load') {
    alert(`${processName}が完了しました。`);
    return;
  }

  if (data.file_name) {
    try {
      // ファイルの中身を受け取る（ログインの Cookie も一緒に送られる）
      const res = await api.get("/api/download_backup", {
        params: { file_name: data.file_name },
        responseType: "blob",                    // 文字ではなく、ファイルの中身として受け取る
      });
      const url = URL.createObjectURL(res.data);  // 受け取った中身に、一時的な住所を付ける
      const link = document.createElement("a");
      link.href = url;
      link.download = data.file_name;             // 保存する時の名前
      link.click();
      URL.revokeObjectURL(url);                   // 一時的な住所を片付ける
      alert(`${processName}が完了し、ダウンロードが開始されました。`);
    } catch {
      alert("ダウンロードに失敗しました。もう一度バックアップからやり直してください。");
    }
  } else {
    alert("処理が完了しました。");
  }
}, [setIsRunning, taskType, processName]);
```

▶ ここで初めて出てくるコード

- `.createObjectURL()`（JavaScript・TypeScript）：`URL.createObjectURL(データ)` は、ブラウザの中にあるデータ（受け取ったファイルの中身など）に、一時的な住所を付ける関数。その住所を `<a>` の `href` にして押すと、ファイルとして保存できます。
- `.revokeObjectURL()`（JavaScript・TypeScript）：`URL.revokeObjectURL(住所)` は、`createObjectURL` で付けた一時的な住所を片付ける関数。使い終わったら呼んで、ブラウザの記憶を空けます。
- `setIsRunning()`（React）：データ管理画面で、「そのバックアップ・ロードが実行中か」を覚えている値を変える関数（`useState` で作った物を渡している）。`setIsRunning(false)` で「終わった」にします。
- `alert()`（JavaScript・TypeScript）：ブラウザに、小さな知らせの窓を出す関数。`alert("処理が完了しました。")`。OK を押すまで、画面の操作が止まります。
- `.href`（JavaScript・TypeScript）：`link.href = url` は、`<a>`（リンク）の行き先の住所を決める書き方です。読む時は、今の行き先の住所の全部が返ります（`location.href` なら、今のページの URL）。

④ テスト: `test_download_file`（2455〜2462 行目）を消し、T-2 の「`media` の外は 404」のテストに置き換えます。画面側のテスト（`frontend/src/tests/AdministratorPage/AdministratorLoading.test.tsx` の 116・157・208・223・238・252・266・280・322 行目）は、`file_path` を `file_name` に変えます。

確かめ方: ✅ 直した写しで、`../../outside.txt` のような名前や、サーバーの中の絶対の場所を渡すと 404、`media` の中のバックアップは受け取れて、受け取った後に消えることを確かめました。手元では、データ管理画面で作業詳細選択肢データ（小さい）をバックアップし、ダウンロードしたファイルが Excel で開けることを確かめます。

4.5 S-5 どのサイトからでも、ログイン中の人として操作できる（CORS・Cookie・CSRF）

🔴 すぐ ／ ✅ 手元で再現

どこ: `hozen_another/settings.py` 53〜59 行目（CORS）、76〜77 行目（Cookie）、Django REST framework の CSRF の扱い

今:（説明のコメントを足しています）

コピー

```
CORS_ORIGIN_ALLOW_ALL = True        # どのサイトにも「API の返事を読んでよい」と答える
CORS_ALLOW_CREDENTIALS = True       # しかも Cookie（ログイン）付きで
CORS_ALLOW_ORIGINS = [              # ← 名前の綴りが違うので、Django は読んでいない（正しくは CORS_ALLOWED_ORIGINS）
    'http://localhost:3000',
    ...
]
SESSION_COOKIE_SAMESITE = 'None'    # 別のサイトから送られた頼みにも、ログインの Cookie を付ける
```

何が困るか: この 3 つと、「Django REST framework が、このアプリのログインでは CSRF を確かめない」ことが重なって、**このアプリにログインしたまま、悪いサイトを開くだけで、そのサイトが裏でこのアプリを操作したり、データを読んだりできます。**

- ✅ 別のサイトを名乗った問い合わせに、サーバーは「そのサイトからの、Cookie 付きの読み取りを許す」と答えました（`Access-Control-Allow-Origin` にそのサイト名、`Access-Control-Allow-Credentials` に `true`）。
- ✅ CSRF トークンを付けずに送った削除が、そのまま通りました。

直し方:

① `settings.py` の 53〜59 行目と 76〜77 行目を、次のようにします。

コピー

```
# CORS: 別のサイトからの読み取りは、手元の開発用の画面（npm start）だけに許す
CORS_ALLOW_CREDENTIALS = True
CORS_ALLOWED_ORIGINS = [
    'http://localhost:3000',
]

# Cookie: 別のサイトから送られた頼みには、ログインの Cookie を付けない
SESSION_COOKIE_SAMESITE = 'Lax'
# https の時だけ Cookie を送る。手元（DEBUG=True）では http なので False になる
SESSION_COOKIE_SECURE = env.bool('COOKIE_SECURE', default=not DEBUG)
CSRF_COOKIE_SECURE = SESSION_COOKIE_SECURE
```

本番では、画面（React）もサーバー（Django）も同じ住所から出ているので、`'Lax'` で困ることはありません。これで、手元で動かす時に 76〜77 行目を書き換える必要も無くなります（D-4、000 巻 §8.1）。

② CSRF の確かめは、S-1 の `LoginNoAuthentication` が、ログイン中の人の書き込みのたびにします。

③ 画面に CSRF トークン（`csrftoken` という Cookie）を配ります。画面（`index.html`）を返す時と、ログインした時に配ります。

コピー

```
# main_views.py
from django.views.decorators.csrf import ensure_csrf_cookie
from django.utils.decorators import method_decorator

# Reactメインページ呼び出し
@ensure_csrf_cookie                    # 画面を開いた時に、csrftoken の Cookie を配る
def react_view(request):
  return render(request, 'index.html')

@method_decorator(ensure_csrf_cookie, name='dispatch')   # ログインした時にも配る（npm start の時のため）
class Login(APIView):
  permission_classes = [AllowAny]
  ...
```

▶ ここで初めて出てくるコード

- `render()`（Django）: Django で、テンプレート（HTML のひな形）に値を入れて、答え（HTML）を作る関数。`render(request, "テンプレート名", {"名前": 値})`。
- `method_decorator()`（Django）: 関数用の印（デコレーター）を、クラスの画面（ビュー）のメソッドにも付けられるようにする物。
- `@ensure_csrf_cookie`（Django）: その画面（ビュー）の返事に、CSRF の合言葉（`csrftoken` というクッキー）を必ず付ける印。React の画面は、このクッキーの値を、書き込みの時に `X-CSRFToken` という見出しに入れて送ります（`frontend/src/api/axios.ts`）。
- `@method_decorator`（Django）: `@method_decorator(ensure_csrf_cookie, name="dispatch")` をクラスの上に付けて、そのクラスの画面に印を付けます。

④ 画面の側は直さなくて大丈夫です。`frontend/src/api/axios.ts` の 5〜6 行目で、すでに「`csrftoken` の Cookie を、`X-CSRFToken` という見出しに入れて送る」設定になっています。

⚠️ **本番に出した直後:** 出す前から開きっぱなしの画面は、まだ CSRF トークンを持っていないので、書き込みが 403 になります。使う人には「一度、画面を再読み込み（F5）してください」と伝えます。

確かめ方: ✅ 直した写しで、別のサイトを名乗った問い合わせに CORS の答えが返らないこと、ログイン中の書き込みが、トークン無しなら 403・トークン付きなら 200 になることを確かめました。

4.6 S-6 ログインにパスワードが無い（従業員番号だけ）

🔴 すぐ（ただし、やり方は決める必要がある） ／ 📖 コードから

どこ: `kosu/views/main_views.py` 79〜102 行目（`Login`）、`hozen_another/settings.py` 74 行目（`SESSION_COOKIE_AGE = 315360000` ＝ 10 年）、`main_views.py` 117 行目（ヘルプの初期化で作る従業員番号 `12345` の管理者）

今: 人員の表にある従業員番号を入れれば、その人としてログインできます（000 巻 §2.2）。管理者の番号を知っていれば管理者として入れます。一度ログインすると、10 年間ログインしたままです。ログインの時に、セッションの番号を新しくしていません（`cycle_key`）。

何が困るか: 従業員番号は、名札・書類・メールなどで、ほかの人も知っている物です。

直し方（どれにするかは、前任者・会社の IT 担当と決める）:

案	中身	手間
A（おすすめ）	**会社の Microsoft アカウント（Entra ID）でサインインする。**Azure App Service の「認証」という機能を有効にすると、アプリの手前で Microsoft のサインインを求めるようになる。アプリは、サインインした人の名前（メールアドレスなど）を、`X-MS-CLIENT-PRINCIPAL-NAME` という見出しで受け取れるので、人員の表にその列を足して結び付ける	会社の Entra ID の管理者の協力が要る。アプリの直しは中くらい
B	**パスワードを足す。**人員の表に `password` の列を足し（Django の `make_password` で、元に戻せない形にして保存）、ログインで `check_password` で比べる。初めのパスワードの配り方と、変える画面を作る	中くらい。migrate が要る
C（すぐできる最低限）	セッションを短くし、ログインのたびにセッションの番号を新しくする。ヘルプの初期化で作る `12345` を、すぐ消す決まりにする	小さい

C は、A・B のどちらにするにしても先にやっておけます。

コピー

```
# main_views.py 90〜91 行目
    if member.objects.filter(employee_no=input_number).exists():
      request.session.cycle_key()                # ログインのたびに、セッションの番号を新しくする
      request.session['login_No'] = input_number
```

▶ ここで初めて出てくるコード

- `.cycle_key()`（Django）：`request.session.cycle_key()` は、セッションの中身はそのままで、セッションの番号（クッキーの値）だけを新しくするメソッド。ログインの時に呼ぶと、ログインの前の番号を知っている人が、なりすませなくなります。

コピー

```
# settings.py 74 行目（例: 12 時間。毎日ログインし直すことになるので、使う人と決める）
SESSION_COOKIE_AGE = 60 * 60 * 12
```

4.7 S-7 ヘルプ画面の「全部消す」API が、パスワードを確かめていない

🔴 すぐ ／ ✅ 手元で再現

どこ: `kosu/views/main_views.py` 106〜171 行目（`Help`）、176〜189 行目（`HelpPass`）、`frontend/src/MainPage/Help.tsx` 32・57 行目

今:

- パスワードを確かめるのは `help_pass` という別の API だけで、実際に消す `help` の API は確かめません。✅ ログインもパスワードも無しで `/api/help/` に送ると、200 が返りました（初期化が動く）。今は画面の側の B-1 のせいで、画面からは動いていません。
- `help_pass` は、何回でも試せます。
- `help_pass` は、頼みのたびに `.env` を読み直し（180〜183 行目）、`==` で比べています。

直し方:

① `settings.py` で、パスワードを 1 回だけ読みます。

コピー

```
HELP_PATH = env('HELP_PATH', default='')       # ヘルプ画面のパスワード（無ければ空 ＝ ヘルプは使えない）
```

② `Help.post` の最初で、パスワードを確かめます。消す処理は「全部うまくいった時だけ残す」（`transaction.atomic`）にします。

コピー

```
import hmac
from django.db import transaction

def help_password_ok(given):
  """ヘルプ画面のパスワードが合っているか（文字の長さで答えの速さが変わらない比べ方）"""
  if not settings.HELP_PATH:
    return False
  return hmac.compare_digest(str(given or '').encode('utf-8'), settings.HELP_PATH.encode('utf-8'))


class Help(APIView):
  permission_classes = [AllowAny]
  # POST処理
  def post(self, request):
    post_data = request.data
    if not help_password_ok(post_data.get('password')):
      return Response({'status': 'error', 'message': 'パスワードが違います。'}, status=status.HTTP_403_FORBIDDEN)
    memberReset = post_data.get('memberReset')
    defReset = post_data.get('defReset')
    settingReset = post_data.get('settingReset')

    try:
      with transaction.atomic():          # この中の変更は、全部うまくいった時だけ残る
        if memberReset:
          ...（今の 116〜147 行目）
        if defReset:
          ...（今の 150〜162 行目）
        if settingReset:
          ...（今の 165〜167 行目）
      return Response({'status': 'success', 'message': 'OK'}, status=status.HTTP_200_OK)
    except Exception:
      return Response({'status': 'error', 'message': 'データの初期化中にエラーが発生しました。'}, status=status.HTTP_400_BAD_REQUEST)
```

▶ ここで初めて出てくるコード

- `.compare_digest()`（Python）：`hmac.compare_digest(a, b)` は、2 つの値が同じかを、かかる時間から中身を推測されない方法で比べる関数です。合言葉やハッシュを比べる時に使います。
- `.atomic()`（Django）：`with transaction.atomic():` の中のデータベースの変更を、「全部成功か、全部取り消し」にする書き方。

> 💡 `hmac.compare_digest` は、`==` と同じく「同じか」を比べますが、どこまで合っていたかで答えの速さが変わりません。文字列のまま渡すと、日本語の入ったパスワードでエラーになるので、`.encode('utf-8')` でバイトにしてから比べます。

③ `HelpPass` に、同じ比べ方と、回数の制限を付けます（1 つの住所から 1 分に 5 回まで。6 回目からは 429 が返る）。

コピー

```
from rest_framework.throttling import SimpleRateThrottle

class HelpPassThrottle(SimpleRateThrottle):
  rate = '5/min'                                    # 1 分に 5 回まで
  def get_cache_key(self, request, view):
    return f'help_pass_{self.get_ident(request)}'  # 送ってきた住所（IP）ごとに数える


class HelpPass(APIView):
  permission_classes = [AllowAny]
  throttle_classes = [HelpPassThrottle]
  def post(self, request):
    if help_password_ok(request.data.get('password')):
      return Response(True, status=status.HTTP_200_OK)
    return Response(False, status=status.HTTP_403_FORBIDDEN)
```

> **▶ ここで初めて出てくるコード**
>
> - **`SimpleRateThrottle`**（Django REST framework）：「決まった時間に、何回まで」と回数を制限する部品の土台。`rate = '5/min'` で 1 分に 5 回、`get_cache_key` で何ごとに数えるか（ここでは送ってきた住所ごと）を決めます。超えると 429 が返ります。

④ 画面（`Help.tsx` 57〜61 行目）：消す API にも、パスワードを一緒に送ります（URL の直しは B-1）。

コピー

```
    await api.post("/api/help/", {
      password,                          // 12 行目の useState で持っている、入力されたパスワード
      memberReset: isMemberReset,
      defReset: isDefReset,
      settingReset: isSettingReset,
    });
```

⑤ 人員の初期化をすると、従業員番号 `12345` の管理者ができます（117 行目）。この番号はコードに書いてあるので、だれでも知り得ます。初期化の後は、**すぐに自分を管理者として登録し、`12345` を消す** ように、画面にも書きます（000 巻 §8.2）。

確かめ方: ✅ 直した写しで、パスワード無しの `/api/help/` が 403 になり人員が消えないこと、間違ったパスワードの 6 回目が 429 になること、日本語のパスワードでも正しく通ることを確かめました。

4.8 S-8 サーバーのログを、ログインしていなくても読める

🔴 すぐ ／ ✅ 手元で再現

どこ: `kosu/views/main_views.py` 935〜953 行目（`WebConsoleLogView`）、`frontend/src/Components/LogConsole.tsx`（5 秒ごとに読む）、`kosu/views/team_views.py` 515・517 行目（`print(team_shop)`）

今: ✅ ログインせずに `/api/web_console_log/` を開くと、ログの中身が返りました。ログには、`print` した物が全部入ります（D-9）。たとえば `team_views.py` の 515・517 行目は、調べ物の名残で、ショップの人の従業員番号の並びを `print` しています。エラーの時は、エラーの文（`str(e)`）をそのまま返しています（953 行目）。

直し方:

コピー

```
class WebConsoleLogView(APIView):                  # View → APIView にして、
  permission_classes = [IsAdministrator]           # 管理者だけにする
  def get(self, request, *args, **kwargs):
    ...（今の 937〜949 行目のまま）
    except Exception:
      views_logger.exception('web_console.log を読めませんでした')   # くわしい中身はログにだけ書く
      return JsonResponse({"error": "ログを読めませんでした。"}, status=500)
```

> **▶ ここで初めて出てくるコード**
>
> - **`.exception()`**（Python）：`logger.exception('…')` は、文と一緒に、今起きたエラーのくわしい中身（どこで起きたか）をログに書くメソッド。`except` の中で使います。

- `team_views.py` の 515・517 行目の `print(team_shop)` を消します。
- ほかにも、エラーの文をそのまま画面に返している所（`asynchronous_views.py` の 61・78 行目などの `f'ファイル書き込みエラー: {str(e)}'`）は、決まった文にして、くわしい中身はログにだけ書きます。

4.9 S-9 リポジトリに、氏名入りのバックアップの Excel が記録されている

🔴 すぐ ／ 📖 コードから（ファイルの見出しだけ確かめた。中身は開いていない）

どこ: リポジトリの直下の 5 つのファイル（コミット `30d45e7`、2026 年 4 月 17 日）と `data.xlsx`

ファイル	入っている物
`人員データバックアップ_20260417.xlsx`	人員（氏名・従業員番号・ショップ・権限）
`問い合わせデータバックアップ_20260417.xlsx`	問い合わせ（従業員番号・氏名・内容・回答）
`班員データバックアップ_20260417.xlsx`	班員（班長と班員の従業員番号）
`工数区分定義データバックアップ_20260417.xlsx`	工数区分定義
`作業詳細選択肢データバックアップ_20260417.xlsx`	作業詳細の選択肢
`data.xlsx`（2025 年 1 月）	工数定義区分・作業詳細の 2 列（D-11 の `generate_prediction` が作る形）

ほかに、Visual Studio の作業用のファイル（`.vs/`。コードの索引のデータベースも入っている）と、テストの記録（`.coverage`）も記録されています（D-3）。

何が困るか: リポジトリを見られる人は、だれでも、これらの Excel を取り出せます。ファイルを消すコミットをしても、**昔のコミット（履歴）には残ります。**

直し方:

1. **GitHub で、リポジトリを見られる人の範囲を確かめます。**リポジトリの Settings → General のいちばん下（Danger Zone）の「Change repository visibility」に、今の範囲が出ます。だれでも見られる（Public）なら、組織（`nomura-shokki`）の管理者と相談して、すぐに Private にします。
2. 会社の決まり（個人情報の扱い）に従って、担当の人に知らせます。
3. ファイルを、記録から外します（手元のファイルは消えません）。

コピー

```
git rm --cached -- "*データバックアップ_*.xlsx" data.xlsx
```

リポジトリの直下の `.gitignore` に、次の行を足して、これからも記録されないようにします。

コピー

```
/*.xlsx
```

4. **履歴からも消します。**GitHub の説明「Removing sensitive data from a repository」の手順（`git filter-repo` という道具を使う）に従います。履歴を書き換えると、全員が clone し直す必要があり、`master` に強制的に push すると Actions が動いて本番に出直します。**必ず前任者・組織の管理者と相談し、時間を決めてから** やります。

5. これからは、データ管理画面でダウンロードしたバックアップを、**リポジトリのフォルダの中に保存しない** でください（ダウンロード先を、リポジトリの外のフォルダにする）。

---

5. 不具合（B）

5.1 B-1 ヘルプ画面の初期化が、URL の `/` が 1 つ多くて動かない

🔴 早めに ／ ✅ 手元で再現

どこ: `frontend/src/MainPage/Help.tsx` 57 行目、`frontend/src/tests/MainPage/Help.test.tsx` 126 行目

今: `api.post("/api/help//", …)` と、最後の `/` が 2 つあります。Django の URL の表（`kosu/urls.py`）に `help//` は無いので、最後の「それ以外は画面を返す」決まり（`hozen_another/urls.py` 14 行目）に回り、CSRF の確かめで断られます。✅ 手元では 403 でした。つまり、**今の本番では、ヘルプ画面の初期化は動いていません。**

画面のテスト（`Help.test.tsx` 126 行目）は、この間違った URL `"/api/help//"` が送られることを確かめているので、テストは通ってしまっています。

直し方: 57 行目を `"/api/help/"` にし、パスワードも送ります（S-7 の④）。`Help.test.tsx` の 126 行目も `"/api/help/"` に直します。

> 💡 前任者の手元には、57 行目だけを直した、まだコミットしていない変更があります。それを使ってもよいですし、ここに書いた形で直しても同じです。

5.2 B-2 バックアップのダウンロードの URL に `//` ができる

🔴 早めに ／ ✅ 手元で再現（URL の行き先まで。本番では確かめていない）

どこ: `frontend/.env.production` 5 行目（本番の行）、`frontend/src/AdministratorPage/AdministratorLoading.tsx` 40 行目

今: `.env.production` の本番の住所は、最後が `/` で終わっています。40 行目は、その後ろに `/api/download_backup` をつなげるので、できる住所は「… `.net//api/download_backup?…`」になります。✅ 手元の Django では、`//api/download_backup` は API の表に当たらず、画面（`index.html`）を返す決まりに回りました。

本番の手前にある Azure の仕組みが `//` を 1 つにまとめていなければ、**ダウンロードしたファイルの中身は、バックアップではなく画面の HTML になります。**

本番での確かめ方: 管理者でデータ管理画面を開き、作業詳細選択肢データ（小さい）をバックアップして、ダウンロードしたファイルが Excel で開けるかを見ます。

直し方: S-4 の③（axios でファイルを受け取る）にすれば直ります。axios は、住所の最後の `/` と、次の頭の `/` を 1 つにまとめてつなぎます（ほかの API が今動いているのは、このためです）。あわせて、D-4 で `.env.production` の住所自体を要らなくします。

5.3 B-3 定義の Ver 名を変える・消すと、その Ver で入れた工数が開けなくなる

🟠 早めに ／ ✅ 手元で再現

どこ: `kosu/models.py` 72 行目（`def_ver2`）、`kosu/views/def_views.py` 164〜179 行目（`DefUpdate.put`）、191〜208 行目（`DefDelete.delete`）

今: 工数の行は、使った定義を **Ver の名前（文字）** で覚えています（`def_ver2`）。定義の編集で名前を変えたり、定義を消したりしても、工数の行の `def_ver2` は元の名前のままです。

✅ 手元で、定義の名前を変えた後、その Ver で入れた工数を開くと「工数区分データが存在しません。」（401）になりました。工数の編集・班員の詳細・集計・班の Excel 出力も、同じ理由で動かなくなります（000 巻 §8.3）。

直し方:

① 名前を変える時は、その Ver を使っている工数の行も、同じ時に書き換えます。

コピー

```
from django.db import transaction
from ..models import Business_Time_graph

  # DefUpdate.put（171〜178 行目を置き換える）
    if serializer.is_valid():
      old_name = def_instance.kosu_name
      new_name = serializer.validated_data.get('kosu_name')
      if new_name != old_name and kosu_division.objects.filter(kosu_name=new_name).exists():
        return Response({'status': 'error', 'message': '入力した工数区分定義Ver名はすでに登録されています。'},
                        status=status.HTTP_400_BAD_REQUEST)
      with transaction.atomic():                  # 定義と工数の書き換えを、まとめて 1 回で
        serializer.save()
        if new_name != old_name:
          Business_Time_graph.objects.filter(def_ver2=old_name).update(def_ver2=new_name)
      if request.session.get('input_def') == old_name:
        request.session['input_def'] = new_name   # 自分のセッションの「使う定義」も新しい名前に
      return Response(serializer.data, status=status.HTTP_200_OK)
```

> **▶ ここで初めて出てくるコード**
>
> - `.validated_data`（Django REST framework）: DRF の `serializer.validated_data` は、確かめ（`is_valid()`）を通った後の、安全な値です。

- `.update()` は、行を 1 つずつ保存しないので速いですが、操作履歴（`History`）には残りません。
- ほかの人のセッションには古い名前が残るので、その人は一度ログインし直すか、定義の Ver を選び直します。

② 消す時は、その Ver を使っている工数がある間は、消せないようにします。最後の 1 つも消せないようにします（消すと、だれもログインできなくなる。000 巻 §8.3）。

コピー

```
  # DefDelete.delete（206 行目と 207 行目の間に足す）
    used = Business_Time_graph.objects.filter(def_ver2=def_instance.kosu_name).count()
    if used:
      return Response({'status': 'error', 'message': f'この Ver で入力した工数が {used} 件あるので、消せません。'},
                      status=status.HTTP_409_CONFLICT)
    if kosu_division.objects.count() <= 1:
      return Response({'status': 'error', 'message': '最後の 1 つの定義は消せません。'}, status=status.HTTP_409_CONFLICT)
```

③ すでに切り離されてしまった工数があるかは、次のように調べられます（読むだけで、何も変えません）。

コピー

```
# python manage.py shell の中で
from kosu.models import Business_Time_graph, kosu_division
names = set(kosu_division.objects.values_list('kosu_name', flat=True))
lost = Business_Time_graph.objects.exclude(def_ver2__in=names).exclude(def_ver2__isnull=True).exclude(def_ver2='')
print(lost.count(), list(lost.values_list('def_ver2', flat=True).distinct()))
```

▶ ここで初めて出てくるコード

- **`.values_list()`**（Django）：問い合わせの結果を、タプル（値の組）の並びで返すメソッド。`flat=True` を付けると、1 つの列の値だけの並びになります。
- **`.exclude()`**（Django）：問い合わせから、条件に合う行を除くメソッド（`filter` の逆）。
- **`.distinct()`**（Django）：問い合わせの結果から、重なった値を 1 つにするメソッド。`.values_list('def_ver2', flat=True).distinct()` は、使われている定義の Ver 名を、重ならないように並べます。

5.4 B-4 班員に入れた人を人員から消すと、班長の画面が 500

🟠 早めに ／ ✅ 手元で再現

どこ: `kosu/views/team_views.py` 295〜297 行目（`TeamCalendar`）、447〜449 行目（`TeamOverTime`）

今: 班のメンバーの名前を、1 人ずつ `member.objects.get(employee_no=m)` で探しています。メンバーのだれかが人員から消されていると、見つからずにエラーになります。✅ 手元で、班のメンバーを人員から消すと、班長の週間カレンダーが 500 になりました。

直し方: まとめて探し、見つからない人は「（削除された人）」と出します。

コピー

```
    # 294〜297 行目（TeamCalendar）と 446〜449 行目（TeamOverTime）を置き換える
    names = {str(m.employee_no): m.name for m in member.objects.filter(employee_no__in=valid_member_numbers)}
    member_name_list = [[m, names.get(str(m), '（削除された人）')] for m in valid_member_numbers]
```

あわせて、退社した人は消さずに、ショップを「異動・退社」にする決まりにすると、記録も残ります（000 巻 §9.2）。

5.5 B-5 人員から消した人の問い合わせが、開けない・直せない・消せない

🟠 早めに ／ ✅ 手元で再現

どこ: `kosu/views/inquiry_views.py` 160〜163 行目（`InquirDetail.get`）、341〜344 行目（`InquirUpdate.put`）、392〜395 行目（`InquirUpdate.delete`）

今: 問い合わせを開く時に、質問した人を `member.objects.get` で探し、見つからないと「質問者の人員情報が見つかりません。」（401）で止まります。✅ 手元で、人員から消した人の問い合わせを開くと、この返事でした（401 なので、画面はログイン画面に飛びます。B-10）。

直し方: 質問した人がいなくても、問い合わせは開けるようにし、その人への通知の処理だけを飛ばします。

コピー

```
# 160〜163 行目を置き換える
inquir_member = member.objects.filter(employee_no=inquir_instance.employee_no2).first()   # いなければ None

# 170 行目
if inquir_member and inquir_member.employee_no == member_data.employee_no:
  ...

# 242 行目
inquir_member_serializer = MemberSerializer(inquir_member, many=False) if inquir_member else None

# 248 行目
  'inquir_member_data': inquir_member_serializer.data if inquir_member_serializer else None,
```

`put`（341〜344 行目）と `delete`（392〜395 行目）も同じく、`.first()` にして、`inquir_member` が `None` の時は、その人の通知を書き換える所を飛ばします。画面（`InquirDetail.tsx`）も、`inquir_member_data` が無い時に「（削除された人）」と出すようにします。

5.6 B-6 同じ従業員番号の人が 2 人いると、その人の画面がほとんど 500

🟠 早めに ／ ✅ 手元で再現

どこ: `kosu/models.py` 23 行目（`member.employee_no`）、70・73 行目（`Business_Time_graph`）、116 行目（`kosu_division.kosu_name`）

今: 従業員番号に「同じ値は 1 つだけ」（unique）の決まりがありません。登録の時に API の中で確かめてはいますが、/admin/ の画面や、ほぼ同時の登録では、同じ番号が 2 つできます。✅ 手元で同じ番号の人を 2 人にすると、その番号でのメインメニューが 500 になりました（`member.objects.get` は、2 つ見つかるとエラー）。

同じように、工数（1 人 1 日に 1 行のはず）と、定義の Ver 名にも決まりが無く、画面に「複数の工数データが存在します。」「複数の工数区分データが存在します。」が出る原因になります。

直し方:

① 先に、本番に重なりが無いかを調べます（読むだけ）。

コピー

```
# python manage.py shell の中で
from django.db.models import Count
from kosu.models import member, Business_Time_graph, kosu_division
print(member.objects.values('employee_no').annotate(n=Count('id')).filter(n__gt=1))
print(Business_Time_graph.objects.values('employee_no3', 'work_day2').annotate(n=Count('id')).filter(n__gt=1))
print(kosu_division.objects.values('kosu_name').annotate(n=Count('id')).filter(n__gt=1))
```

▶ ここで初めて出てくるコード

- `.values()` （Django）：行を、モデルではなく、決めた列だけの辞書で受け取るメソッド。`member.objects.values('employee_no').annotate(n=Count('id'))` は、「従業員番号ごとに、何行あるか」を数える時の書き方です。
- `Count` （Django）：Django の `Count("id")` は、ORM の集計で、数を数えます。`.annotate(n=Count("id"))` で、グループごとの件数が付きます。

重なりがあれば、どちらを残すかを使う人と決めて、先に直します（重なりがあると、③の migrate が失敗します）。

② モデルに決まりを足します。

コピー

```
class member(models.Model):
  ...
  employee_no = models.IntegerField('従業員番号', unique=True)        # 同じ番号は 1 人だけ


class Business_Time_graph(models.Model):
  ...
  class Meta:
    constraints = [
      # 1 人 1 日に 1 行だけ
      models.UniqueConstraint(fields=['employee_no3', 'work_day2'], name='kosu_one_row_per_day'),
    ]


class kosu_division(models.Model):
  kosu_name = models.CharField('工数区分定義Ver名', blank=True, null=True, max_length=100, unique=True)
  ...
```

▶ ここで初めて出てくるコード

- `.UniqueConstraint()` （Django）：「この組み合わせの値は、表の中で重ならない」という決まりを表す物。モデルの `class Meta:` の `constraints` に書きます。`fields=['employee_no3', 'work_day2']` は「同じ人の、同じ就業日の行は、2 つ作れない」。`name=` は、決まりの名前（表の中で重ならない名前）です。

③ `python manage.py makemigrations` で migration のファイルを作り、手元で `migrate` とテストを流してから、本番の migrate の手順（09 巻 §14.3、D-2）で本番に入れます。

5.7 B-7 設定データが無いと、多くの画面が 500

🟠 早めに ／ 📖 コードから

どこ: `administrator_data.objects.order_by("id").last()` を `try` 〜 `except administrator_data.DoesNotExist` で囲んでいる 10 か所（`main_views.py` 224〜227・434〜437・471〜474・491〜494 行目、`team_views.py` 498〜501 行目、`inquiry_views.py` 107〜110・165〜168・286〜289・335〜338・397〜400 行目）

今: `.last()` は、データが無い時にエラーを出さず、`None` を返します。なので `except administrator_data.DoesNotExist` は決して動かず、その後の `admin_data.pop_up1` などで別のエラー（`AttributeError`）になり、500 になります。設定データは、次の時に無くなります。

- 設定データのロードの途中で失敗した時（`tasks.py` 990 行目で、先に全部消してから作るため。B-8）
- /admin/ の画面で消した時

直し方:「無ければ、最初の値で作って返す」関数を 1 つ作り、10 か所をそれに置き換えます。

コピー

```
# kosu/utils/main_utils.py に足す
def get_settings():
  """設定データ（administrator_data）を 1 件返す。無ければ、最初の値で作る。"""
  settings_row = administrator_data.objects.order_by('id').last()
  if settings_row is None:
    settings_row = administrator_data.objects.create(menu_row='20')   # 一覧の件数の初期値（ヘルプの初期化と同じ）
  return settings_row
```

コピー

```
# 使う所（例: main_views.py 224〜227 行目を置き換える）
    admin_data = get_settings()
```

5.8 B-8 データのロードが途中で失敗すると、データが半分だけ消える

🟠 早めに ／ 📖 コードから

どこ: `kosu/tasks.py` の `load_choice_file`（735〜743 行目）、`load_setting_file`（990〜1010 行目）、`load_def_file`（630〜648 行目）、`load_kosu_file`（197〜235 行目）

今: ロードは「先に消す → 1 行ずつ足す」の順で、**途中で失敗しても、消した物は戻りません。**たとえば作業詳細選択肢のロードは、最初に全部消してから足すので、Excel の途中の行に変な値があると、選択肢が途中までしか無い状態になります。

直し方:「消す → 足す」を `transaction.atomic()` で囲み、全部うまくいった時だけ残るようにします。一時ファイルの片付けは `finally` にまとめます。

コピー

```
from django.db import IntegrityError, transaction


def load_choice_file(file_path):
  try:
    wb = openpyxl.load_workbook(file_path)
    ws = wb.worksheets[0]
    expected_headers = ['定義記号', '作業詳細']
    actual_headers = [ws.cell(1, col).value for col in range(1, len(expected_headers) + 1)]
    if actual_headers != expected_headers:
      return {'status': 'error', 'message': '無効なファイルフォーマットです。'}, None

    with transaction.atomic():            # この中の変更は、全部うまくいった時だけ残る
      def_choice.objects.all().delete()
      for i in range(2, ws.max_row + 1):
        def_choice.objects.create(def_symbol=ws.cell(row=i, column=1).value,
                                  def_select=ws.cell(row=i, column=2).value)
    return {'status': 'success'}, None
  except Exception as e:
    return {'status': 'error', 'message': str(e)}, None
  finally:
    if os.path.exists(file_path):          # うまくいっても、失敗しても、一時ファイルを消す
      os.remove(file_path)
```

▶ ここで初めて出てくるコード

- **`finally`**（Python）: `try … except` のあと、成功しても失敗しても最後に必ず動く所。

1 行ずつ `IntegrityError` を受け止めて飛ばしている物（`load_member_file` など）は、その 1 行を、もう 1 つ内側の `transaction.atomic()` で囲みます。囲まずに受け止めると、外側の取り消しの仕組みが壊れて、次の行から全部失敗します。

コピー

```
    with transaction.atomic():
      for i in range(2, ws.max_row + 1):
        try:
          with transaction.atomic():       # この 1 行だけの区切り。失敗しても、この行だけ取り消す
            member.objects.create(...)
        except IntegrityError as create_e:
          print(f'{i} 行目を飛ばしました: {create_e}')   # 今のコードと同じく print（D-9 で logging に）
```

5.9 B-9 ログインが切れていると、401 ではなく 500 になる API がある

🟡 余裕のある時に ／ 📖 コードから

どこ: `kosu_views.py` 148 行目（`KosuNew.post`）、324 行目（`OverTime.post`）、676 行目（`KosuUpdate.put`）、`def_views.py` 157 行目（`DefUpdate.get`）、`team_views.py` 82〜87 行目（`TeamNew.post`）

今: 最初に「ログインしているか」を確かめずに `member.objects.get(employee_no=login_no)` などをしているので、ログインが切れていると 500 になります。

直し方: S-1 を入れると、中身が動く前に 401 が返るので、自動で直ります。

5.10 B-10 ログイン以外の理由でも 401 を返し、画面がログイン画面へ飛ぶ

🟠 早めに ／ ✅ 手元で再現

どこ: 401 を「ログインしていない」以外の意味で使っている 29 か所

返事の文	場所
工数区分データが存在しません。／複数の工数区分データが存在します。	`kosu_views.py` 95・627・629・634・636・1180・1259 行目、`def_views.py` 281・336 行目、`team_views.py` 217・224 行目
工数データが見つかりません。	`kosu_views.py` 798・801 行目
設定データが見つかりません。	`main_views.py` 227・437・474・494 行目、`team_views.py` 501 行目、`inquiry_views.py` 110・168・289・338・400 行目
質問者の人員情報が見つかりません。	`inquiry_views.py` 163・344・395 行目
工数データのユーザーが存在しません。／メンバーが存在しません。（他人の工数の持ち主）	`main_views.py` 661 行目、`team_views.py` 208 行目
複数の班員データが存在します。	`team_views.py` 32 行目

今: 画面の多くは「401 ならログイン画面へ移る」作りです。✅ 手元で、定義の名前を変えた後に工数を開くと 401 が返りました。使う人から見ると「急にログイン画面に戻された」になり、何が起きたか分かりません。

直し方: 401 は「ログインしていない」の時だけにします。見つからない時は 404、データが食い違っている時は 409（Conflict）にし、画面では返事の `message` を出します。「設定データが見つかりません。」は B-7 で無くなります。

5.11 B-11 作業詳細選択肢の「ロード開始」が、定義のロード中に「実行中...」になる

🟡 余裕のある時に ／ 📖 コードから

どこ: `frontend/src/AdministratorPage/AdministratorLoading.tsx` 494 行目

今: `value={runningStates.DefLoad ? "実行中..." : "ロード開始"}` と、定義（Def）のロードの状態を見ています（454 行目からの写し間違い）。

直し方: `runningStates.ChoiceLoad` にします。

5.12 B-12 ファイル選びで CSV・XLS も選べるが、サーバーは xlsx しか読めない

🟡 余裕のある時に ／ 📖 コードから

どこ: `AdministratorLoading.tsx` の `accept=".csv, .xlsx, .xls"`（398・438・478・518・558・598・638 行目）と「ファイルを選択 (CSV/XLSX)」（408・448・488・528・568・608・648 行目）

今: サーバーのロード（`kosu/tasks.py`）は `openpyxl.load_workbook` で読むので、xlsx しか読めません。CSV や XLS を選ぶと、ロードがエラーになります。

直し方: 7 か所とも `accept=".xlsx"` と「ファイルを選択 (XLSX)」にします。

5.13 B-13 バックアップ・ロードがサーバーの再起動で止まり、「待ち」のまま残る

🔴 早めに ／ 📖 コードから

どこ: `kosu/views/asynchronous_views.py` 26 行目（`AsyncTask` を作る所）、191〜192 行目（`threading.Thread`）、`hozen_another/settings.py` 125〜131 行目（`Q_CLUSTER`）、`AdministratorLoading.tsx` 63〜79 行目（1 秒ごとの問い合わせ）

今:

- バックアップ・ロードは、Web のサーバーの中で、別の **スレッド**（同じプログラムの中で並んで動く流れ）として動きます。デプロイや Azure の再起動でサーバーが止まると、途中のタスクも止まり、記録は「pending（待ち）」のまま残ります。画面は 1 秒ごとに問い合わせ続け、終わりません。
- `Q_CLUSTER`（django-q という、別の手順でタスクを動かす部品の設定）はありますが、使っていません（D-8）。
- 26 行目で、日付などを確かめる **前に** タスクの記録を作るので、日付の間違いで断られた時にも、「pending」の記録が残ります。

直し方:

① 26 行目の `AsyncTask.objects.create(...)` を、191 行目（スレッドを始める所）のすぐ上に移します。

② 画面（63〜79 行目）で、決めた時間（例: 30 分 = `Q_CLUSTER` の `timeout` の 1800 秒）を過ぎたら問い合わせをやめ、「時間内に終わりませんでした。タスクの一覧を見てください」と出します。

③ 長い目で見ると、タスクは Web のサーバーとは別の手順（django-q の `qcluster` など）で動かすのが安全です。本番で `qcluster` を動かしているか、前任者に確かめてから決めます（000 巻 §10）。

5.14 B-14 月で絞り込むと、月初めの 0〜9 時の記録が前の月に入る

🟡 余裕のある時に ／ 📖 コードから

どこ: `kosu/views/main_views.py` 748 行目（タスクの一覧）、858 行目（操作履歴の一覧）

今: `created_at__startswith=search_day[:7]` のように、日時を **文字にして頭の 7 文字（例: `2026-11`）** で比べています。日時はデータベースに世界標準時（UTC。日本より 9 時間遅い）で入っていて、文字にすると UTC のまま比べるので、日本時間の 11 月 1 日 0〜9 時の記録は「10 月」に入ります。

直し方: 年と月で比べます。`USE_TZ = True`（`settings.py` 154 行目）なので、Django が日本時間（`TIME_ZONE = 'Asia/Tokyo'`）に直してから比べます。

コピー

```
    if mode == 'month':
      year, month = search_day[:7].split('-')
      tasks = tasks.filter(created_at__year=int(year), created_at__month=int(month))
```

（工数の就業日 `work_day2` は日付だけで時刻が無いので、今のままで大丈夫です。）

5.15 B-15 思っていない値が送られると 500 になる

🟡 余裕のある時に ／ 📖 コードから

どこ:

場所	今
`kosu_views.py` 1222〜1246 行目（`KosuTotal.post`）	`period` が「日間・月間・年間」のどれでもないと、`kosu_data` が作られずにエラー
`team_views.py` 334〜338 行目（`TeamCalendarWeekJump`）	`week` が `A`・`B` のどちらでもないと、`date_object` が作られずにエラー
`kosu_views.py` 912〜913 行目（`KosuCalendarChange`）	年・月を確かめずにセッションへ入れるので、次に開いた時（891 行目など）にエラー
`kosu_views.py` 162〜163・805〜806 行目	`time1`・`time2` が無いと、日時に直す所でエラー

直し方: 受け取った値を、使う前に確かめて、おかしければ 400 で返します。

コピー

```
period = request.data.get('period')
if period not in ('日間', '月間', '年間'):
  return Response({'status': 'error', 'message': '期間の指定が正しくありません。'}, status=status.HTTP_400_BAD_REQUEST)
```

5.16 B-16 月間・年間の集計で、最初の 1 件の定義 Ver で全部を数える

🟡 余裕のある時に ／ 📖 コードから

どこ: `kosu/views/kosu_views.py` 1249〜1257 行目（`KosuTotal.post`）

今: 月間・年間の集計では、その期間の工数のうち **最初の 1 件** の定義 Ver を選び、全部をその定義で数えます。定義を新しくした月のように、期間の中に Ver が 2 つ以上あると、もう一方の Ver の工数も、別の Ver の作業名で数えられます（Ver ごとに、記号 A・B… が指す作業が違うため）。

直し方: 使う人と相談して決めます。すぐできるのは「この期間には定義の Ver が 2 つ以上あります」と画面に出すことです。きちんと直すなら、Ver ごとに分けて集計します。

コピー

```
    versions = list(kosu_data.exclude(def_ver2__isnull=True).exclude(def_ver2='')
                    .values_list('def_ver2', flat=True).distinct())
    if len(versions) > 1:
      response_data['warning'] = f'この期間には定義の Ver が {len(versions)} つあります（{"・".join(versions)}）。'
```

（`response_data` は 1273 行目で作っているので、その後ろに置きます。画面の `KosuTotal.tsx` で `warning` を出します。）

5.17 B-17 操作履歴を 1 件保存するたびに、全件を数えている

🟡 余裕のある時に ／ 📖 コードから

どこ: `kosu/models.py` 432〜441 行目（`History.save`）、391〜401 行目（`AsyncTask.save`）、`kosu/signals.py`（人員・工数などを保存・削除するたびに `History` を 1 件作る）

今: `History` を保存するたびに、表の全件数を数え（`count()`）、上限（50 万件）を超えた分を 1 件ずつ消しています。工数を 1 件保存するたびに、50 万件近い表を数えることになります。工数の一括削除（`tasks.py` 157〜163 行目）では、消す 1 件ごとに `History` ができるので、件数が多いととても遅くなります。

直し方: 数えるのを 1,000 件に 1 回にし、消すのも 1 回でまとめて消します。

コピー

```
def save(self, *args, **kwargs):
  super().save(*args, **kwargs)
  if self.id % 1000 == 0:                       # 1,000 件に 1 回だけ、古い物を片付ける
    newest = (self.__class__.objects.order_by('-id')
              .values_list('id', flat=True)[self.MAX_RECORDS - 1:self.MAX_RECORDS])
    if newest:                                  # 新しい方から数えて 50 万件目より古い物を、まとめて消す
      self.__class__.objects.filter(id__lt=newest[0]).delete()
```

▶ ここで初めて出てくるコード

- **`"-列の名前"`**（Django）: Django の `ordering = ["-created_at"]` や `.order_by("-created_at")` の、名前の頭の `-` は、「大きい順（新しい順）に並べる」という意味です。`-` が無ければ、小さい順（古い順）です。
- **`%`**（Python）: Python の `7 % 2` は、割った余り（`1`）です。

5.18 B-18 問い合わせの通知が、別の人の問い合わせを指すことがある

🟡 余裕のある時に ／ 📖 コードから

どこ: `kosu/views/inquiry_views.py` 112 行目

今: 新しい問い合わせを保存した後、通知に付ける番号を「いちばん新しい問い合わせ」（`inquiry_data.objects.order_by('id').last()`）から取っています。2 人がほぼ同時に送ると、片方の通知が、もう一方の人の問い合わせを指します。

直し方: 112 行目を消し、121 行目の `inquiry_data_id.id` を、保存したばかりの `new_inquiry.id` にします。

5.19 B-19 従業員番号を変えると、その人の工数・班・問い合わせが切り離される

🟠 早めに ／ 📖 コードから

どこ: `kosu/views/member_views.py` 188〜203 行目（`MemberUpdate.put`）

今: 人員の編集で、従業員番号も変えられます。ところが、工数（`Business_Time_graph.employee_no3`）・班（`team_member.employee_no5`・`member1` 〜 `member15`）・問い合わせ（`inquiry_data.employee_no2`）・設定の問い合わせ担当者（`administrator_employee_no1` 〜 `3`）は、どれも番号そのもので人を指しているので、番号を変えると、その人の過去の工数などが「だれの物でもない」状態になります。

直し方（どちらかを選ぶ）:

- **番号は変えられないようにする（おすすめ）。**番号を間違えて登録した時は、新しく登録し直します。

コピー

```
# MemberUpdate.put の 195 行目（data = request.data）の下に足す
if str(data.get('employee_no')) != str(pk):
  return Response({'status': 'error', 'message': '従業員番号は変えられません。新しく登録してください。'},
                  status=status.HTTP_400_BAD_REQUEST)
```

画面（`MemberEdit.tsx`）の従業員番号の欄も、書き換えられないようにします。

- **番号を変えたら、上の 4 つの表も同じ時に書き換える。**`transaction.atomic()` の中で、4 つの表の番号を `.update()` します。

5.20 B-20 `.gitattributes` のせいで、Excel・フォントなどのファイルが壊れる

🟠 早めに ／ ✅ 手元で再現

どこ: リポジトリの直下の `.gitattributes` 1〜4 行目

コピー

```
# 行末記号をWindows(CRLF)で明示
* text eol=crlf
*.png binary
*.ico binary
```

今: 「全部のファイルを文字のファイルとして扱い、改行を CRLF（Windows の改行）にする」設定です。`.png` と `.ico` 以外のバイナリ（文字ではない）ファイルも、文字のファイルとして扱われ、中の「改行に見えるバイト」が書き換えられて壊れます。

- ✅ `git check-attr` で調べると、フォント（`.woff`）や圧縮ファイル（`.tar.xz`）も「文字のファイル」の扱いでした。
- ✅ リポジトリに記録されている Excel 5 つのうち 3 つは、取り出しても Excel として開けませんでした。
- 手元で `git status` に、`.gz`・`.woff`・`.ttf`・`.tar.xz` が「変更あり」と出続けるのも、このためです。
- GitHub Actions も、この設定のとおりに取り出してから本番に送る物を作ります。

直し方: 文字のファイルかどうかを Git に判断させ、バイナリのファイルは書き換えないようにします。

コピー

```
# 文字のファイルかどうかは Git に判断させ、文字のファイルだけ、改行を CRLF にする
* text=auto eol=crlf
# 中身がバイナリのファイル（改行を書き換えない）
*.png binary
*.ico binary
*.jpg binary
*.gif binary
*.xlsx binary
*.gz binary
*.xz binary
*.woff binary
*.woff2 binary
*.ttf binary
*.eot binary
*.db binary
*.sqlite binary
```

書き換えたら、記録されているファイルの扱いを、新しい設定に合わせ直してコミットします。

コピー

```
git add --renormalize .
git status
git commit -m "B-20: バイナリのファイルの改行を書き換えないようにする"
```

> **▶ ここで初めて出てくるコード**
>
> - **`git add --renormalize`**（Git）：`git add --renormalize .` は、記録されているファイルを、今の `.gitattributes` の決まり（改行の扱いなど）で記録し直すコマンド。`.gitattributes` を変えた後に使います。

- すでに壊れたファイルは、元には戻りません。`staticfiles/` のフォントなどは、D-3 でリポジトリから外せば、Actions が毎回作り直します。
- Linux で動かすシェルのファイル（`.sh`）を足す時は、`*.sh text eol=lf` も足します（CRLF だと Linux で動きません）。

5.21 B-21 Excel の見出し「ポップアップID6」の中身が ID5

🟡 余裕のある時に ／ 📖 コードから

どこ: `kosu/tasks.py` 275 行目（人員のバックアップ）、341 行目（人員のロード）、903 行目（設定のバックアップ）、979 行目（設定のロード）

今: 見出しが「ポップアップID6」ですが、入っているのは `pop_up_id5` です。バックアップとロードで同じ見出しを使っているので、動きは正しいです。

直し方: 直すなら、4 か所を同時に「ポップアップID5」にし、ロードの側では、古いバックアップ（「ポップアップID6」）も読めるように、どちらの見出しも受け付けるようにします。急ぎません。

---

6. 開発・運用の整理（D）

6.1 D-1 GitHub Actions が、テストを流さずに本番へ出している

🟠 早めに ／ 📖 コードから

どこ: `.github/workflows/master_hozen-kosu-react.yml`

今: `master` に push されると、テストを流さずに、build して本番に出します。テストが落ちる変更でも、そのまま本番に出ます。ほかに、

- `npm install`（31〜32 行目）は、`package-lock.json` と違う版の部品を入れることがあります。`npm ci` は、`package-lock.json` のとおりに入れます。
- Secrets（秘密の値）を、全部の手順から見える場所（10〜17 行目の `env`）に置いています。`npm install` で入れる部品の中のプログラムからも見えます。なお、本番のアプリは App Service の環境変数を読むので、`DB_NAME` などの Secrets は、今の手順では使われていません（`collectstatic` に要るのは `SECRET_KEY` と `DEBUG` だけ）。
- Node.js 20（28 行目）は、2026 年 4 月でサポートが終わりました（D-8）。

直し方: テストの手順を、build と本番に出す手順の前に入れます。Pull Request の時も、テストだけ流します（本番には出さない）。

コピー

```yaml
name: Deploy Django + React to Azure
on:
  push:
    branches:
      - master
  pull_request:            # Pull Request の時も、テストまで流す（本番には出さない）
  workflow_dispatch:
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "22"
      - name: Install React dependencies
        run: |
          cd frontend
          npm ci
      - name: Test React
        run: |
          cd frontend
          npm run test:run
      - name: Build React
        run: |
          cd frontend
          npm run build
      - name: Install Python dependencies
        run: |
          python -m venv antenv
          source antenv/bin/activate
          pip install --upgrade pip
          pip install -r requirements.txt
      - name: Test Django
        env:
          SECRET_KEY: test-only
          DEBUG: "False"
          HELP_PATH: test-only
        run: |
          source antenv/bin/activate
          python manage.py test --settings=hozen_another.test_settings
      - name: Collect static files
        env:
          SECRET_KEY: ${{ secrets.SECRET_KEY }}
          DEBUG: ${{ secrets.DEBUG }}
        run: |
          source antenv/bin/activate
          python manage.py collectstatic --noinput
      - name: Create deployment zip
        if: github.event_name != 'pull_request'
        run: |
          zip -r release.zip . -x ".git/*" "node_modules/*" "frontend/node_modules/*"
      - name: Deploy to Azure Web App
        if: github.event_name != 'pull_request'
```

```
      uses: azure/webapps-deploy@v3
      with:
        app-name: hozen-kosu-React
        publish-profile: ${{ secrets.AZUREAPPSERVICE_PUBLISHPROFILE_054C4F3FA5C14FFF913C35ACF9E2E28A }}
        package: release.zip
```

▶ ここで初めて出てくるコード

- `name:`（GitHub Actions）：`name:` は、名前です。いちばん上の `name: Deploy Django + React to Azure` は、ワークフロー（自動の手順）の名前、`- name: Build React` は、手順 1 つの名前で、GitHub の Actions の画面に表示されます。
- `on:`（GitHub Actions）：`on:` は、「いつ動かすか」です。`push:`（送った時）・`pull_request:`（プルリクエストの時）・`schedule:`（決まった時刻）などを並べます。
- `push:`（GitHub Actions）：`push:` は、「リポジトリに送った（push）時に動かす」指定です。
- `branches:`（GitHub Actions）：`branches:` の下に、ブランチの名前を並べて、「そのブランチに送った時だけ」としぼります。このアプリは `- master` なので、`master` に push した時だけ動きます。
- `pull_request:`（GitHub Actions）：`pull_request:` は、「プルリクエスト（変更を取り込んでほしいというお願い）が作られた・更新された時に動かす」指定です。
- `workflow_dispatch:`（GitHub Actions）：`on:` の下の `workflow_dispatch:` は、「GitHub の Actions の画面の『Run workflow』ボタンで、手で動かせるようにする」指定です。
- `jobs:`（GitHub Actions）：`jobs:` の下に、動かす仕事（ジョブ）を並べます。ジョブは、ふつう同時に動きます。
- `build-and-deploy:`（GitHub Actions）：`jobs:` の下の `build-and-deploy:` は、仕事（ジョブ）の名前です。名前は自分で決めます。このアプリは、build・テスト・本番に出す、を 1 つの仕事でやっています。
- `runs-on:`（GitHub Actions）：`runs-on: ubuntu-latest` は、その仕事を、どの OS の機械で動かすかです。GitHub が用意した Linux（Ubuntu）の機械を、その時だけ借ります。
- `steps:`（GitHub Actions）：`steps:` の下に、その仕事の手順を、上から順に `-` で並べます。
- `-`（GitHub Actions）：YAML の行の頭の `-` は、「並びの 1 つ」を表します。`steps:` の下の `- name: …` は、手順の 1 つ目・2 つ目…です。字下げ（空白の数）で、どこに入るかが決まるので、そろえて書きます。
- `uses:`（GitHub Actions）：`uses: actions/checkout@v4` は、ほかの人が作った決まった手順（アクション）を使う指定です。`actions/checkout` は、リポジトリのファイルを取ってくる手順、`@v4` はその版です。`actions/setup-python` は Python を、`actions/setup-node` は Node.js を入れる手順です。
- `with:`（GitHub Actions）：`with:` は、`uses:` で使うアクションに渡す設定です。
- `python-version:`（GitHub Actions）：`actions/setup-python` に渡す設定で、入れる Python の版です。`"3.12"` と引用符で囲むのは、`3.10` が数の `3.1` と読まれるような間違いを防ぐためです。
- `node-version:`（GitHub Actions）：`actions/setup-node` に渡す設定で、入れる Node.js の版です。`"22"` なら、22 系の新しい版が入ります。
- `run:`（GitHub Actions）：`run:` は、その手順で動かすコマンドです。`run: |` と書くと、次の行から何行も書けます（`cd frontend` の次の行に `npm ci`、のように）。
- `|`（GitHub Actions）：`run: |` の `|` は、「次の行から、字下げした所までを、改行も含めてそのまま 1 つの文字にする」という印です。コマンドを何行も書く時に使います。
- `env:`（GitHub Actions）：`env:` の下に `名前: 値` を並べると、その手順（または仕事全体）の中だけで使える環境変数になります。手順の下に書けば、その手順の中だけで見えます。

- **`SECRET_KEY:`**（GitHub Actions）：`env:` の下の `SECRET_KEY: …` は、`SECRET_KEY` という環境変数を作る行です。Django の `settings.py` は、起動の時にこの値を読みます（無いと動きません）。
- **`DEBUG:`**（GitHub Actions）：`env:` の下の `DEBUG: "False"` は、`DEBUG` という環境変数を作る行です。`"False"` と引用符で囲むのは、YAML が `False` を「文字」ではなく「真偽の値」と読まないようにするためです。
- **`HELP_PATH:`**（GitHub Actions）：`env:` の下の `HELP_PATH: …` は、ヘルプ画面のパスワードを入れる環境変数を作る行です。テストの手順では、テストのためだけの値を入れます。
- **`${{ … }}`**（GitHub Actions）：`${{ }}` の中に書いた物は、GitHub Actions が、動かす時に値に置き換えます。`${{ secrets.SECRET_KEY }}` は、リポジトリの Secrets に入れた `SECRET_KEY` の値、`${{ github.event_name }}` は、何をきっかけに動いたか（`push`・`pull_request` など）です。
- **`secrets.名前`**（GitHub Actions）：`secrets.SECRET_KEY` のように書くと、リポジトリの Settings → Secrets and variables → Actions に入れた、秘密の値を読めます。値は画面やログには出ません（`***` になります）。
- **`if:`**（GitHub Actions）：`if:` に条件を書くと、その条件が合う時だけ、その手順を動かします。`if: github.event_name != 'pull_request'` は「Pull Request がきっかけでない時だけ」（＝ master に push した時だけ本番に出す）。
- **`app-name:`**（GitHub Actions）：`azure/webapps-deploy` に渡す設定で、送り先の Azure App Service の名前です（このアプリは `hozen-kosu-React`）。
- **`publish-profile:`**（GitHub Actions）：`azure/webapps-deploy` に渡す設定で、Azure に送るための「通行証」（発行プロファイル）です。値は秘密なので、Secrets から読みます。
- **`package:`**（GitHub Actions）：`azure/webapps-deploy` に渡す設定で、送るファイルです。このアプリは、全部を 1 つにまとめた `release.zip` を送ります。
- **`npm ci`**（Node.js）：`package-lock.json` に書かれたとおりの版で、部品を入れ直すコマンド（`node_modules` を一度消してから入れる）。`npm install` と違い、`package-lock.json` を書きかえないので、いつ・どこで入れても同じになります。
- **`source`**（WSL・bash）：`source antenv/bin/activate` は、Python の仮想環境（`antenv`）に入るコマンド（Linux・Mac の書き方）。入ると、`python`・`pip` が、その仮想環境の物になります。
- **`pip install --upgrade`**（Python の道具）：`pip install --upgrade pip` は、部品をいちばん新しい版にするコマンド。ここでは `pip` 自身を新しくしています。
- **`python --noinput`**（Django の道具）：`python manage.py collectstatic --noinput` の `--noinput` は、「上書きしてよいか」などの質問をせずに進める、という指定です。人が答えられない GitHub Actions の中で使います。
- **`zip`**（WSL・bash）：ファイルやフォルダを、1 つの zip ファイルにまとめるコマンド。`zip -r release.zip .` は、今いるフォルダの中を全部、`release.zip` にまとめます。
- **`zip -r`**（WSL・bash）：`zip` の `-r` は、フォルダの中のフォルダまで、全部まとめるという指定です（recursive）。
- **`zip -x`**（WSL・bash）：`zip` の `-x` の後ろに書いた物は、まとめる物から外します。`-x ".git/*" "node_modules/*"` は、Git の記録と部品のフォルダを入れない、という意味です。

- 画面側のテスト（`npm run test:run`）が今通るかは、確かめていません。先に手元で流し、通ってから「Test React」の手順を足します。
- Node.js を 22 にする前に、手元の Node.js 22 で `npm ci` と `npm run build` が通ることを確かめます。
- テストの手順の `SECRET_KEY` などは、テストのためだけの値です（本物の秘密ではない）。

6.2 D-2 本番のデータベースの migrate を、だれが・どこで打つか決まっていない

🔴 早めに ／ 📖 コードから

今: Actions は migrate をしません（09 巻 §14.1）。モデルを変えた時（B-6 など）は、本番のデータベースに migrate を流す必要があります。

直し方: 前任者に、今までどうしていたかを確かめ（000 巻 §10）、決まった手順にして 09 巻 §14.3 に書き足します。よくあるやり方は次の 2 つです。

やり方	中身	気を付けること
App Service のスタートアップ コマンドで流す	アプリが起動するたびに `python manage.py migrate` をしてから `gunicorn` を起動する	失敗するとアプリが起動しない。時間のかかる migrate では起動が遅れる
手で流す	App Service の「SSH」（Azure の画面から入れる、サーバーの中の黒い画面）で `python manage.py migrate` を打つ	打ち忘れ。必ずバックアップの後に

6.3 D-3 リポジトリに、入れなくてよい物が 1,200 ファイル以上ある

🟡 余裕のある時に ／ 📖 コードから

今: 次の物が記録されています（2026 年 11 月 9 日に `git ls-files` で数えた）。

物	数	なぜ要らないか
`staticfiles/`	995	Actions の `collectstatic` が毎回作る
`frontend/build/`	20	Actions の `npm run build` が毎回作る
`node_modules/`・`package.json`・`package-lock.json`（リポジトリの直下）	171＋2	画面の部品は `frontend/` の中の物を使う。直下の物は使っていない（同じ部品が `frontend/package.json` にある）
`frontend/node-v22.17.1-linux-x64.tar.xz`	1（約 30 MB）	Node.js の本体。Actions は `setup-node` で入れる
`.vs/`	13	Visual Studio の作業用のファイル
`.vscode/`・`.coverage`・`frontend/package.json.save`・`web_console.log`	5	個人の設定・テストの記録・書きかけの写し・ログ（`.vscode/` は `.gitignore` に書いてあるのに、その前に記録された）
バックアップの Excel・`data.xlsx`	6	S-9

直し方: リポジトリの直下の `.gitignore` に足し、記録から外します（手元のファイルは消えません）。

コピー

```
/staticfiles/
/frontend/build/
/node_modules/
/package.json
/package-lock.json
/frontend/*.tar.xz
/.vs/
/.coverage
/web_console.log*
/media/
/*.xlsx
*.save
```

コピー

```
git rm -r --cached staticfiles frontend/build node_modules package.json package-lock.json
git rm --cached frontend/node-v22.17.1-linux-x64.tar.xz .coverage frontend/package.json.save web_console.log
git rm -r --cached .vs .vscode
git commit -m "D-3: 作れる物・個人の物をリポジトリから外す"
```

▶ ここで初めて出てくるコード

- **`git rm -r`**（Git）：`git rm` の `-r` は、フォルダの中を全部、という指定です。`git rm -r --cached staticfiles` は、`staticfiles` フォルダの中を全部、Git の記録から外します（手元のファイルは残る）。

- 外した後の最初のデプロイで、本番の画面と、管理画面（/admin/）の見た目が崩れていないかを確かめます。
- 手元で Django だけを動かす時、`frontend/build/` が無いと、`localhost:8000` を開いた時の画面（`index.html`）が出ません。画面は `npm start`（`localhost:3000`）で見ます（01 巻）。
- 記録から外しても、昔の履歴には残ります。リポジトリの大きさを小さくしたい時は、S-9 の 4 と同じ手順になります。

6.4 D-4 手元で動かす時に、2 か所を書き換える必要がある

🟡 余裕のある時に ／ 📖 コードから

今: 手元で動かす時は、`settings.py` の 76〜77 行目と、`frontend/.env.production` を書き換え、コミットの前に戻す決まりです（000 巻 §8.1）。戻し忘れると、本番が壊れます。

直し方: 書き換えなくて済むようにします。

① `settings.py`：S-5 の①にすると、`.env` の `DEBUG=True`（手元）なら `Secure` が自動で `False` になります。

② 画面（`frontend/src/api/axios.ts`）：API の住所を、画面と同じ住所（空の文字）にします。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
import axios from "axios";
const api = axios.create({
  baseURL: process.env.REACT_APP_API_BASE_URL || "",   // 空なら、画面と同じ住所の /api/… に送る
  withCredentials: true,
  xsrfCookieName: "csrftoken",
  xsrfHeaderName: "X-CSRFToken",
});
export default api;
```

`frontend/.env.production` の `REACT_APP_API_BASE_URL` の行は、全部消します（ファイルごと消してもよい）。本番では、画面もサーバーも同じ住所なので、これで動きます。ステージング用に build し直す必要も無くなります。

③ 手元（`npm start` の `localhost:3000`）のために、`frontend/package.json` の最初の方に 1 行足します。

コピー

```
  "proxy": "http://localhost:8000",
```

▶ ここで初めて出てくるコード

- **`"proxy":`**（設定ファイル（JSON））：`frontend/package.json` の `"proxy": "http://localhost:8000"` は、`npm start` で動く開発用のサーバーに、「自分で答えられない頼み（`/api/…` など）は、`localhost:8000` の Django に回して」と伝える設定です。本番の build には関係しません。

`npm start` で動く開発用のサーバーが、`/api/…` への頼みを、`localhost:8000` の Django に回してくれます。ブラウザから見ると、画面と API が同じ住所になるので、CORS の設定も要らなくなります（その後、`corsheaders` を外せます。D-7）。

- この回し方は、`<a href>` で開くような「画面としての移動」は回しません。ダウンロードを axios にする（S-4 の③）のは、このためでもあります。
- 直したら、手元で `npm start` → ログイン → 工数の入力 → データ管理のバックアップ、まで動くことを確かめます。

6.5 D-5 住所（ホスト名）がコードに直接書いてある

🟡 余裕のある時に ／ 📖 コードから

どこ: `hozen_another/settings.py` 19〜23 行目（`ALLOWED_HOSTS`）、162〜166 行目（`CSRF_TRUSTED_ORIGINS`）

今: 本番の住所がコードに書いてあり、ステージングの住所はありません（09 巻 §2）。住所が変わるたびに、コードを直して出し直す必要があります。

直し方: 環境変数から読みます。

コピー

```
ALLOWED_HOSTS = env.list('ALLOWED_HOSTS', default=['localhost', '127.0.0.1'])
CSRF_TRUSTED_ORIGINS = env.list('CSRF_TRUSTED_ORIGINS', default=['http://localhost:8000', 'http://127.0.0.1:8000'])
```

> **▶ ここで初めて出てくるコード**
>
> - `.list()`（Django）：`env.list('ALLOWED_HOSTS', default=[...])` は、`django-environ` で、`a,b,c` のようにカンマで区切った環境変数を、並び（リスト）として読むメソッド。

> ⚠️ **順番が大事です。**このコードを本番に出す **前に**、App Service の環境変数に `ALLOWED_HOSTS`（本番のホスト名）と `CSRF_TRUSTED_ORIGINS`（`https://` ＋ 本番のホスト名）を入れておきます。先にコードを出すと、本番のすべての頼みが 400 になります。値は、今の 19〜23 行目と 162〜166 行目に書いてある本番の住所です。

6.6 D-6 HTTPS まわりの安全設定が足りない

🟡 余裕のある時に ／ 📖 コードから

今: `settings.py` に、https で動いていることを Django に教える設定などがありません。

直し方: Azure の App Service の「HTTPS のみ」がオンになっているかを確かめ（Azure の画面の「構成」→「全般設定」。前任者か Azure の権限を持つ人と）、`settings.py` に足します。

コピー

```
# Azure の手前の仕組みが https を受けた時に付ける見出しを見て、「https で来た」と分かるようにする
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
# ブラウザに「このサイトはいつも https で開いて」と覚えさせる秒数（初めは 0。慣れたら 3600 → 31536000 と上げる）
SECURE_HSTS_SECONDS = env.int('SECURE_HSTS_SECONDS', default=0)
```

> **▶ ここで初めて出てくるコード**
>
> - `.int()`（Django）：`env.int('名前', default=0)` は、`django-environ` で、環境変数を整数として読むメソッド。無い時は `default` の値になります。

`SECURE_HSTS_SECONDS` を大きくすると、ブラウザはその間 http で開けなくなるので、小さい値から始めます。

6.7 D-7 使っていない部品が 10 個以上入っている

🟡 余裕のある時に ／ 📖 コードから（2026 年 11 月 9 日に `git grep` で、コードの中に名前が出てこないことを確かめた）

どこ: `requirements.txt`、`hozen_another/settings.py` 32〜33・93〜95・107〜109 行目

部品	何をする物か	今
`xlwings`	Windows・Mac の Excel を動かす	使っていない（Linux の本番では使えない）
`python-Levenshtein`・`beautifulsoup4`	文字の近さを測る・HTML を読む	使っていない
`django-crispy-forms`・`django-widget-tweaks`・`django-widgets-improved`・`django-filter`	Django の画面（テンプレート）のフォームを作る	使っていない（今の画面は React）
`python-dotenv`・`python-json-logger`	`.env` を読む・ログを JSON で書く	使っていない（`.env` は `django-environ` が読む）
`pandas`	表の計算	使っていない関数（D-11 の `generate_prediction`）でだけ使っている
`django-bootstrap4`・`django-bootstrap-datepicker-plus`	Django の画面の見た目	`INSTALLED_APPS`（32〜33 行目）と `TEMPLATES` の `builtins`（93〜95 行目）、`BOOTSTRAP4`（107〜109 行目）にだけ出てくる。昔の Django の画面の名残

直し方: 1 つずつ外して、`pip install -r requirements.txt` → テスト → 手元で動かす、を繰り返します。`django-bootstrap4`・`django-bootstrap-datepicker-plus` は、`settings.py` の 3 か所も一緒に消します。`corsheaders` は、D-4 の後に外せます。

6.8 D-8 サポート（安全の直し）が終わった部品を使っている

🟠 早めに ／ 📖 コードから

部品	今の版	状況（2026 年 11 月）	上げる先
Django	4.2.7	4.2 系のサポートは 2026 年 4 月で終わった。安全の直しが出ない	まず 4.2 系の最後の版に上げ、次に 5.2（LTS。2028 年 4 月まで）
djangorestframework	3.14.0	Django 5.2 に対応していない	Django 5.2 に対応した版（3.16 以降）
django-q	1.3.9	開発が止まっている（後を継いだのは django-q2）	B-13 と合わせて決める
Node.js（Actions）	20	2026 年 4 月でサポート終了	22 か 24（D-1）
react-scripts（Create React App）	5.0.1	2025 年 2 月に開発の終わりが発表された	Vite（テストは、すでに Vitest と `frontend/vite.config.ts` を使っている）

Django を 5.1 以上にする時は、`settings.py` 168 行目の `STATICFILES_STORAGE` が使えなくなるので、次のように書き換えます。

コピー

```
STORAGES = {
    "default": {"BACKEND": "django.core.files.storage.FileSystemStorage"},
    "staticfiles": {"BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage"},
}
```

進め方: 一度に全部上げません。1 つずつ、テスト → 手元で動かす → （あれば）ステージング → 本番、の順にします。上げる前には、その部品の「リリースノート」（版ごとの変更点）の、変わった所・消えた所を読みます。

6.9 D-9 `print` の行き先のすり替えと、ログのファイルの場所

🟡 余裕のある時に ／ 📖 コードから

どこ: `kosu/views/main_views.py` 27〜38 行目（`PrintToLogger`）、`hozen_another/settings.py` 188 行目、`main_views.py` 938 行目

今:

- `sys.stdout = PrintToLogger()` で、`main_views.py` が読み込まれた瞬間から、**サーバー全体の `print`** がログのファイルに書かれます。調べ物で書いた `print` も、全部ログに残ります（S-8）。
- ログのファイルは、書く側（`settings.py` 188 行目）が「起動した場所からの `web_console.log`」、読む側（938 行目）が「プロジェクトの直下の `web_console.log`」です。起動の場所が違うと、書く所と読む所がずれます。

- `web_console.log` はリポジトリに記録されている（空のファイル）ので、デプロイのたびに上書きされ、ログが消えます。大きさも 1 MB ＋ 予備 1 つまでです。

直し方:

- `print` をやめ、`views_logger.info(...)` などで書きます。そのうえで `PrintToLogger` の 27〜38 行目を消します。
- `settings.py` 188 行目を `'filename': os.path.join(BASE_DIR, 'web_console.log'),` にします。
- `web_console.log` をリポジトリから外します（D-3）。
- 長く残したいログは、App Service のログの機能を使います（前任者・Azure の権限を持つ人と）。

6.10 D-10 古い入口 `/BA...` から、全部の API が使える

🟡 余裕のある時に ／ ✅ 手元で再現

どこ: `hozen_another/urls.py` 11 行目

今: `path('BA', include('kosu.urls'))` があるので、全部の API が、`/api/login/` のほかに `/BAlogin/` のような住所でも動きます。✅ 手元で `/BAlogin/` がログインの API に当たりました。画面からは使っていません。

直し方: 11 行目を消します。

6.11 D-11 使っていないコードが残っている

🟡 余裕のある時に ／ 📖 コードから

場所	中身	消す時に気を付けること
`main_views.py` 41〜47 行目	`get_logs`（どの URL にもつながっていない）	無い
`tasks.py` 78〜144 行目	`generate_prediction`（どこからも呼ばれていない）と、それだけが使う `pandas`	`import pandas as pd`（5 行目）も消す（D-7）
`models.py` 405〜415 行目	`Operation_history`（どこからも使われていない表）	消すと migrate で表が消える。本番の表に中身が無いかを先に確かめる

6.12 D-12 同じコードの繰り返しが多い

🟡 余裕のある時に ／ 📖 コードから

同じことを何か所にも書いていると、1 か所だけ直し忘れます（S-2 の確かめ漏れも、これが原因の 1 つです）。

繰り返し	場所	まとめ方
「ログインしているか・権限があるか」の 5〜10 行	ほぼ全部の API（約 60 か所）	S-1 の後に消す
班の `member1` 〜 `member15` から番号を並べる	`team_views.py` 40〜49・132〜141・284〜293・436〜445 行目、`main_views.py` 348〜357 行目	下の `team_member_numbers`
休憩時間の初めの値	`main_views.py` 122〜145 行目、`member_views.py` 92〜143 行目	1 つの辞書（定数）にする
通知（ポップアップ）を前に詰める	`inquiry_views.py` 183〜197・215〜229・412〜425・436〜448 行目	関数にする
5 つのメニューの API	`main_views.py` 242〜411 行目	ほぼ同じ。1 つの関数にまとめられる
バックアップの 19 個の `if`	`asynchronous_views.py` 36〜189 行目	「URL の名前 → 関数」の表（辞書）にする

コピー

```
# kosu/utils/team_utils.py に足す
def team_member_numbers(team):
  """班（team_member）の member1〜member15 から、空でない従業員番号を並べて返す"""
  if team is None:
    return []
  numbers = [getattr(team, f'member{i}') for i in range(1, 16)]
  return [n for n in numbers if n not in (None, '')]
```

6.13 D-13 URL の名前の入れ違い・末尾の `/` のばらつき

🟡 余裕のある時に ／ 📖 コードから

どこ: `kosu/urls.py` 32〜33 行目、63 行目、99〜100 行目

- 32〜33 行目: `day_update/` の名前が `'item_delete'`、`item_delete/` の名前が `'day_update'` と、入れ違っています。
- 63 行目: 名前が `'team_shop_select/'` と、最後に `/` が付いています。
- 99〜100 行目: `check_backup_status` と `download_backup` だけ、URL の最後に `/` がありません（ほかは全部ある）。

今は、名前（`name=`）を使っている所が無いので、動きには関係しません。近くを直す時に、そろえておきます。99〜100 行目の `/` を足す時は、画面（`AdministratorLoading.tsx` 61 行目など）の URL も同時に直します。

7. テスト（T）

7.1 T-1 直すと落ちるテストが 8 個ある

🔴（安全の直しと一緒に） ／ ✅ 手元で再現

S-1・S-4・S-7 を入れると、今のテストのうち、次の 8 個が落ちます（2026 年 11 月 9 日、直した写しで 331 個を流して、323 個 OK・8 個 失敗）。どれも、ログインせずに呼んでいるか、パスワードを送っていないテストです。

テスト（`kosu/tests/test_backend.py`）	行	直し方
`test_help_post_member_reset`	2248	送る値に `'password'` を足す（テストでは `HELP_PATH` を決めた値にしておく）
`test_help_post_no_reset`	2254	同上
`test_check_task_status_success`	2423	最初に `self.login_as_admin()` を足す。返事は `file_path` ではなく `file_name` を確かめる
`test_check_task_status_error`	2432	最初に `self.login_as_admin()` を足す
`test_check_task_status_pending`	2440	同上
`test_check_task_status_no_id`	2447	同上
`test_check_task_status_invalid_id`	2451	同上
`test_download_file`	2455	**消す。** `media` の外のファイルをダウンロードできることを「正しい」としている。T-2 のテストに置き換える

テストの中で `HELP_PATH` を決めた値にするには、`django.test.override_settings` を使います。

コピー

```
from django.test import override_settings

    @override_settings(HELP_PATH='test-help')
    def test_help_post_member_reset(self):
        response = self.client.post('/api/help/', {'password': 'test-help', 'memberReset': True}, format='json')
        self.assertEqual(response.status_code, 200)
```

> **▶ ここで初めて出てくるコード**
>
> - **`override_settings()`**（Django）：`from django.test import override_settings` で読み込む、テストの間だけ設定を変える道具。`@override_settings(…)` の形で使います。
> - **`@override_settings`**（Django）：テストのメソッドの上に付けると、そのテストの間だけ、`settings.py` の値を変えられる印。`@override_settings(HELP_PATH='test-help')` 。

画面側のテストも、`frontend/src/tests/MainPage/Help.test.tsx` 126 行目（B-1）と、`frontend/src/tests/AdministratorPage/AdministratorLoading.test.tsx` の `file_path`（S-4 の④）を直します。

7.2 T-2 直した所を守るテストを足す

🟠 早めに ／ ✅ 手元で再現（下の 5 個は、直した写しで全部通り、今の `master` では全部落ちることを確かめた）

直した所が、後でうっかり元に戻らないように、テストを足します。`kosu/tests/test_security.py` を新しく作ります。

コピー

```
import json
from rest_framework.test import APIClient
from kosu.models import member, Business_Time_graph
from kosu.tests.test_backend import BaseTestCase


class TestSecurity(BaseTestCase):
    """ログイン・権限・持ち主・CSRF の確かめが効いていることを確かめるテスト"""

    NEW_MEMBER = {'employee_no': 1, 'name': 'x', 'shop': 'P', 'authority': True, 'administrator': True}

    def test_member_new_needs_login(self):
        # ログインしていない → 401、人員はできない
        response = self.client.post('/api/member_new/', self.NEW_MEMBER, format='json')
        self.assertEqual(response.status_code, 401)
        self.assertFalse(member.objects.filter(employee_no=1).exists())

    def test_member_new_needs_authority(self):
        # 「権限」の無い人 → 403
        self.login_as_normal()
        response = self.client.post('/api/member_new/', self.NEW_MEMBER, format='json')
        self.assertEqual(response.status_code, 403)

    def test_cannot_delete_others_kosu(self):
        # 他人の工数は「見つからない」（404）にして、消さない
        others = Business_Time_graph.objects.create(employee_no3=12345, name=self.member, work_day2='2026-01-05')
        self.login_as_normal()
        response = self.client.delete(f'/api/kosu_delete/{others.id}/')
        self.assertEqual(response.status_code, 404)
        self.assertTrue(Business_Time_graph.objects.filter(id=others.id).exists())

    def test_csrf_is_checked_when_logged_in(self):
        # ブラウザと同じく CSRF を確かめるクライアントで、トークン無しの書き込み → 403
        client = APIClient(enforce_csrf_checks=True)
        client.post('/api/login/', json.dumps({'employee_no': 99999}), content_type='application/json')
        response = client.post('/api/set_day/', {'day': '2026-01-05'}, format='json')
        self.assertEqual(response.status_code, 403)
        # ログインの返事で配られた csrftoken を付ければ通る
        token = client.cookies['csrftoken'].value
        response = client.post('/api/set_day/', {'day': '2026-01-05'}, format='json', HTTP_X_CSRFTOKEN=token)
        self.assertEqual(response.status_code, 200)

    def test_download_only_inside_media(self):
        # media の外を指す名前 → 404
        self.login_as_admin()
        response = self.client.get('/api/download_backup', {'file_name': '../manage.py'})
        self.assertEqual(response.status_code, 404)
```

▶ ここで初めて出てくるコード

- `.cookies`（Django）：テスト用のクライアントの `client.cookies` は、そのクライアントが覚えているクッキーです。`client.cookies['csrftoken'].value` で、配られた CSRF の合言葉の値が取れます。

- `BaseTestCase`（`test_backend.py` 38〜74 行目）は、管理者（12345）と一般の人（99999）を作り、`login_as_admin`・`login_as_normal` でログインできるようにしてあるテストの土台です（12 巻）。
- 今のテストは `APIClient()` を使っていて、CSRF を確かめません（テストのための決まり）。CSRF を確かめたい時だけ、`APIClient(enforce_csrf_checks=True)` を使います。

- S-2 の表の API ごとに、「ログインしていない → 401」「権限が無い → 403」「他人の行 → 404」のテストを、同じ形で足していきます。

---

8. 直す順番（おすすめ）



図2: 直す順番のおすすめ。左ほど急ぐ。どの段も「作業用のブランチで直す → テスト → 確かめてから master へ」（master に入れると、そのまま本番に出るため）

段	直す物	目安
1（今すぐ）	S-4 ダウンロード → S-1 共通の確かめ → S-2 確かめ漏れ → S-5 CORS・Cookie・CSRF、T-1・T-2。S-9 の 1・2（リポジトリの公開範囲と、会社への連絡）は、コードを直すより先に	数日
2（1〜2週）	S-3 他人のデータ、S-7 ヘルプ、S-8 ログ、B-1・B-2、S-6 の C（セッションを短く）	1〜2週
3（1か月）	B-3〜B-8、B-10・B-13・B-19・B-20、D-1・D-2	1か月
4（余裕の時）	残りの B・D、D-8 部品を新しく、S-6 の A か B（ログインの方式）	数か月

- 段 1 は、全部まとめて 1 回で本番に出すのがおすすめです。S-1 と S-5 は、片方だけだと画面が動かなくなる組み合わせがあるためです（たとえば CSRF の確かめだけ入れて、トークンを配らないと、書き込みが全部 403 になる）。
- どの段も、§3 の手順（作業用のブランチ → テスト → 手元で確かめる → Pull Request → master）で出します。
- 出した後は、本番で、ログイン・工数の入力・人員の一覧・データ管理のバックアップとダウンロードを、一通り確かめます（09 巻 §14.4）。

---

9. この巻で確かめたこと

9.1 確かめ方

- GitHub の `master`（コミット `30d45e7`）を、`git archive` で手元の別のフォルダに書き出しました（アプリのフォルダ・本番・GitHub には何も書き込んでいない）。
- アプリに入っているテスト用の設定（`hozen_another/test_settings.py`。データベースはメモリの中だけの SQLite）で、Django のテスト用のクライアントを使って、API に頼みを送りました。
- 次に、この巻の S-1・S-4・S-5・S-7 のコードと、S-3 の①（工数の削除）を写しに入れて、同じことを確かめました。
- **本番のサーバー・データベースには、一切触っていません。**本番で同じことを試さないでください。

9.2 結果

確かめたこと	今の `master`	この巻のコードを入れた写し
ログインせずに、人員を「管理者」で登録	201（できた）。その番号でログインもできた	401（できない）
ログインせずに、他人の人員を「管理者」に	200（できた）	401
ログインせずに、設定を変える	200	401
ログインせずに、定義の登録・名前の変更・選択肢の追加	200・200・201	401
ログインせずに、ショップの工数入力状況の Excel を取り出す	200（Excel が返った。先に `def_ver` を送った場合）	401
ログインせずに、他人の工数の就業日を変える	200	401
ログインせずに、操作履歴を消す・問い合わせを消す	200・204	401
ログインせずに、サーバーのログを読む	200	401
ログインせずに、工数の一括削除を始める（日付なし）	400（日付の確かめまで進んだ）	401
ログインせずに、`media` の外のファイルをダウンロード	200（中身が返った）	401（管理者でも 404）
ログインせずに、パスワード無しでヘルプの初期化	200	403
ヘルプの間違ったパスワードを 6 回	—	403 が 5 回、6 回目は 429
一般の人が、他人の工数を消す（CSRF トークン無し）	204（消えた）	403（CSRF）
一般の人が、他人の工数を消す（CSRF トークン付き）	204（消えた）	404（消えない）
一般の人が、他人の問い合わせに回答を書く（CSRF トークン無し）	200（書けた）	403（CSRF）。S-3 の④は写しに入れていない
別のサイトを名乗った CORS の問い合わせ	そのサイト名と `true` が返った	返らない
画面と同じ `/api/help//` への送信	403（ヘルプの初期化は動かない）	（B-1 で直す）
`//api/download_backup` の行き先	画面を返す決まり（`react_view`）	（B-2 で直す）
`/BAlogin/` の行き先	ログインの API	（D-10 で直す）

確かめたこと	今の `master`	この巻のコードを入れた写し
定義の名前を変えた後、その Ver の工数を開く	401「工数区分データが存在しません。」	（B-3 で直す）
班員を人員から消した後、班のカレンダー	500	（B-4 で直す）
人員から消した人の問い合わせを開く	401「質問者の人員情報が見つかりません。」	（B-5 で直す）
同じ番号の人が 2 人の時、メインメニュー	500	（B-6 で直す）
アプリのテスト（331 個）	全部 OK	323 個 OK、8 個 失敗（T-1）
T-2 のテスト（5 個）	5 個とも失敗（問題を見つけた）	5 個とも OK

10. この巻のまとめ

- いちばん大きな問題は、**API ごとに確かめを書いていて、書き忘れが多い** ことです。共通の確かめを 1 つ作って全部の API にかける（S-1）と、今ある漏れが一度にふさがり、これから API を足す時も安全な側になります。
- **今すぐ** 直すのは、ダウンロード（S-4）、共通の確かめ（S-1・S-2）、CORS・Cookie・CSRF（S-5）です。リポジトリの Excel（S-9）は、コードより先に、公開範囲の確認と会社への連絡をします。
- 不具合（B）は、使う人が困っている物（B-1〜B-6・B-10）から直します。
- どれも、**作業用のブランチで直し、テストを流し、手元で確かめてから** master に入れます。master に入れた物は、そのまま本番に出ます。
- 直したら、この巻の該当する行に「済み（日付・コミット）」と書き足しておくと、次の人が助かります。